

ĐẠI HỌC QUỐC GIA TP HCM  
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN  
KHOA CÔNG NGHỆ THÔNG TIN



---

## BÁO CÁO LAB 3

Chủ đề: Advanced MapReduce & Spark Structured APIs

---

Môn học: Nhập môn Dữ liệu lớn

*Sinh viên thực hiện:*

23120099 - Lê Xuân Trí

23120180 - Nguyễn Lê Trung Trực

23120166 - Trần Hữu Kim Thành

23120185 - Nguyễn Hồ Anh Tuấn

*Giảng viên hướng dẫn:*

Thầy Trần Huy Bàn

Thầy Võ Nhật Tân

Ngày 19 tháng 5 năm 2026

# Mục lục

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Thông tin nhóm &amp; Phân công công việc</b>                | <b>1</b>  |
| <b>2</b> | <b>Giới thiệu</b>                                              | <b>1</b>  |
| 2.1      | Bối cảnh bài lab . . . . .                                     | 1         |
| 2.2      | Bộ dữ liệu Amazon Sale Report . . . . .                        | 1         |
| 2.3      | Môi trường thực thi . . . . .                                  | 2         |
| 2.4      | Lý do chọn Scala . . . . .                                     | 2         |
| 2.5      | Tổng quan kết quả đầu ra . . . . .                             | 3         |
| 2.6      | Cấu trúc báo cáo . . . . .                                     | 3         |
| <b>3</b> | <b>Task 1.1: Sliding Window – Most Bought Size per State</b>   | <b>3</b>  |
| 3.1      | Phân tích truy vấn (Query Analysis) . . . . .                  | 3         |
| 3.2      | Phân rã thành các bước cơ bản (Decomposition) . . . . .        | 4         |
| 3.3      | Lý do phân rã . . . . .                                        | 5         |
| 3.4      | Chiến lược thực thi & Triển khai mã nguồn . . . . .            | 5         |
| 3.5      | Phân tích chuyên sâu . . . . .                                 | 7         |
| 3.5.1    | Pseudocode tổng quan . . . . .                                 | 7         |
| 3.5.2    | Tại sao naive nested-loop bất khả thi . . . . .                | 7         |
| 3.5.3    | Key design giải quyết vấn đề thế nào . . . . .                 | 8         |
| 3.5.4    | Xử lý unseen timestamps trong output . . . . .                 | 8         |
| 3.6      | Kết quả thực thi & Xuất dữ liệu . . . . .                      | 8         |
| 3.7      | Minh chứng bằng hình ảnh . . . . .                             | 9         |
| <b>4</b> | <b>Task 1.2: State-level Median Variety per Month</b>          | <b>11</b> |
| 4.1      | Phân tích truy vấn (Query Analysis) . . . . .                  | 11        |
| 4.2      | Phân rã thành các bước cơ bản (Decomposition) . . . . .        | 11        |
| 4.3      | Lý do phân rã . . . . .                                        | 13        |
| 4.4      | Chiến lược thực thi & Triển khai mã nguồn . . . . .            | 13        |
| 4.5      | Phân tích chuyên sâu . . . . .                                 | 16        |
| 4.5.1    | Pseudocode tổng quan . . . . .                                 | 16        |
| 4.5.2    | Tại sao median KHÔNG thể tính bằng Combiner đơn giản . . . . . | 17        |

|          |                                                                              |           |
|----------|------------------------------------------------------------------------------|-----------|
| 4.5.3    | Chiến lược tính median trong MapReduce context . . . . .                     | 17        |
| 4.5.4    | Cách xác định “size $\geq$ XXL” — parsing logic . . . . .                    | 17        |
| 4.6      | Kết quả thực thi & Xuất dữ liệu . . . . .                                    | 18        |
| 4.7      | Minh chứng bằng hình ảnh . . . . .                                           | 19        |
| <b>5</b> | <b>Task 2.1: Percentage of Cancelled Orders per City</b>                     | <b>21</b> |
| 5.1      | Phân tích truy vấn (Query Analysis) . . . . .                                | 21        |
| 5.2      | Phân rã thành các bước cơ bản (Decomposition) . . . . .                      | 22        |
| 5.3      | Lý do phân rã . . . . .                                                      | 24        |
| 5.4      | Chiến lược thực thi & Triển khai mã nguồn . . . . .                          | 24        |
| 5.5      | Phân tích chuyên sâu — Kế hoạch thực thi (Execution Plan Analysis) . . . . . | 27        |
| 5.5.1    | Output của explain(true) . . . . .                                           | 27        |
| 5.5.2    | Phân tích Physical Join Strategy . . . . .                                   | 27        |
| 5.5.3    | Đếm Exchange Nodes (Shuffle) . . . . .                                       | 28        |
| 5.5.4    | Đếm Stages . . . . .                                                         | 29        |
| 5.5.5    | Thảo luận Adaptive Query Execution (AQE) . . . . .                           | 30        |
| 5.6      | Kết quả thực thi & Xuất dữ liệu . . . . .                                    | 30        |
| 5.7      | Minh chứng bằng hình ảnh . . . . .                                           | 32        |
| <b>6</b> | <b>Task 2.2: StdDev of Amount by SKU-Month with Percentile Threshold</b>     | <b>34</b> |
| 6.1      | Phân tích truy vấn (Query Analysis) . . . . .                                | 34        |
| 6.2      | Phân rã thành các bước cơ bản (Decomposition) . . . . .                      | 35        |
| 6.3      | Lý do phân rã . . . . .                                                      | 36        |
| 6.4      | Chiến lược thực thi & Triển khai mã nguồn . . . . .                          | 37        |
| 6.5      | Phân tích chuyên sâu . . . . .                                               | 39        |
| 6.5.1    | Thuật toán bên trong approx_percentile . . . . .                             | 39        |
| 6.5.2    | Exact percentile: sort + row_number . . . . .                                | 39        |
| 6.5.3    | Khi nào approx_percentile đủ tốt, khi nào exact bắt buộc . . . . .           | 40        |
| 6.6      | So sánh hai phương pháp Percentile . . . . .                                 | 40        |
| 6.7      | Thảo luận về phân mảnh dữ liệu (Partitioning Strategy) . . . . .             | 41        |
| 6.8      | Kết quả thực thi & Xuất dữ liệu . . . . .                                    | 42        |
| 6.9      | Minh chứng bằng hình ảnh . . . . .                                           | 43        |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>7 Kết luận</b>                                               | <b>45</b> |
| 7.1 Tổng kết kết quả . . . . .                                  | 45        |
| 7.2 Trade-off giữa MapReduce và Spark Structured APIs . . . . . | 46        |
| 7.3 Bài học rút ra . . . . .                                    | 46        |
| 7.4 Khó khăn và hướng cải thiện . . . . .                       | 47        |
| <b>8 Phụ lục</b>                                                | <b>48</b> |
| 8.1 Source code và Demo . . . . .                               | 48        |
| <b>Tài liệu tham khảo</b>                                       | <b>49</b> |

# 1 Thông tin nhóm & Phân công công việc

| MSSV     | Họ và Tên            | Vai trò     | Công việc                                         | Mức độ hoàn thành |
|----------|----------------------|-------------|---------------------------------------------------|-------------------|
| 23120099 | Lê Xuân Trí          | Nhóm trưởng | Thực hiện Task 1-1 (MapReduce) & Tổng hợp báo cáo | 100%              |
| 23120180 | Nguyễn Lê Trung Trực | Thành viên  | Thực hiện Task 1-2 (MapReduce)                    | 100%              |
| 23120166 | Trần Hữu Kim Thành   | Thành viên  | Thực hiện Task 2-1 (Spark Structured API)         | 100%              |
| 23120185 | Nguyễn Hồ Anh Tuấn   | Thành viên  | Thực hiện Task 2-2 (Spark Structured API)         | 100%              |

## 2 Giới thiệu

### 2.1 Bối cảnh bài lab

Lab 03 của môn Nhập môn Dữ liệu lớn yêu cầu sinh viên giải quyết bốn bài toán phân tích dữ liệu phân tán trên bộ dữ liệu **Amazon Sale Report**, sử dụng hai nhóm công nghệ:

- **Task 1-1 và Task 1-2:** Advanced MapReduce — triển khai thuật toán bằng Spark RDD transformations theo tư duy key-value processing.
- **Task 2-1 và Task 2-2:** Spark Structured APIs — sử dụng DataFrame/Dataset API với Catalyst optimizer.

### 2.2 Bộ dữ liệu Amazon Sale Report

Dataset chứa thông tin đơn hàng trên nền tảng Amazon tại thị trường Ấn Độ, với khoảng **128,000** dòng và **24 cột**. Các cột chính được sử dụng trong bốn task:

| Cột                | Kiểu    | Mô tả                                     |
|--------------------|---------|-------------------------------------------|
| Date               | String  | Ngày đặt hàng, format MM-dd-yy            |
| Status             | String  | Trạng thái đơn (Shipped, Cancelled, ...)  |
| Fulfilment         | String  | Đơn vị thực hiện (Amazon, Merchant)       |
| ship-service-level | String  | Mức dịch vụ (Standard, Expedited)         |
| ship-city          | String  | Thành phố giao hàng                       |
| ship-state         | String  | Bang giao hàng                            |
| SKU                | String  | Mã sản phẩm (Stock Keeping Unit)          |
| Style              | String  | Mã kiểu dáng sản phẩm                     |
| Size               | String  | Size sản phẩm (S, M, L, XL, XXL, ...)     |
| Qty                | Integer | Số lượng mua                              |
| Amount             | Double  | Giá trị đơn hàng (INR)                    |
| promotion-ids      | String  | Danh sách mã khuyến mãi (comma-separated) |
| Courier Status     | String  | Trạng thái vận chuyển                     |

## 2.3 Môi trường thực thi

| Thành phần   | Chi tiết                             |
|--------------|--------------------------------------|
| Apache Spark | 3.4.0                                |
| Scala        | 2.13.8                               |
| Chế độ chạy  | Local mode ( <code>local[*]</code> ) |
| Hệ điều hành | Linux native trên máy local          |
| Java         | OpenJDK 8 (1.8.0_482)                |

## 2.4 Lý do chọn Scala

Scala được chọn làm ngôn ngữ triển khai cho toàn bộ bốn task vì ba lý do chính:

1. **Full điểm ngôn ngữ:** Theo rubric chấm điểm, sử dụng Scala cho giải pháp hoàn chỉnh được 0.125 điểm/task.
2. **Type-safety:** Scala là ngôn ngữ tĩnh kiểu, giúp phát hiện lỗi tại compile-time thay vì runtime — đặc biệt quan trọng khi làm việc với Dataset API.

3. **Native Spark API:** Spark được viết bằng Scala; các API như `aggregateByKey`, `reduceByKey`, Dataset encoders hoạt động tự nhiên nhất với Scala, tránh overhead của PySpark serialization.

## 2.5 Tổng quan kết quả đầu ra

| Task     | Framework            | Output           | Trạng thái                |
|----------|----------------------|------------------|---------------------------|
| Task 1.1 | MapReduce (RDD)      | Task_1-1.csv     | Hoàn thành, đã kiểm chứng |
| Task 1.2 | MapReduce (RDD)      | Task_1-2.csv     | Hoàn thành, đã kiểm chứng |
| Task 2.1 | Spark Structured API | Task_2-1.parquet | Hoàn thành, đã kiểm chứng |
| Task 2.2 | Spark Structured API | Task_2-2.parquet | Hoàn thành, đã kiểm chứng |

## 2.6 Cấu trúc báo cáo

Báo cáo được tổ chức thành các phần sau: thông tin nhóm (Section 1), sau đó mỗi task là một section riêng biệt (Section 2–5) với cấu trúc thống nhất gồm: phân tích truy vấn, phân rã bước, lý do phân rã, chiến lược thực thi kèm code snippets, phân tích chuyên sâu, kết quả và kiểm chứng. Phần kết luận (Section 6) tổng kết trade-off giữa MapReduce và Spark Structured APIs.

# 3 Task 1.1: Sliding Window – Most Bought Size per State

## 3.1 Phân tích truy vấn (Query Analysis)

Bài toán yêu cầu xác định size được mua nhiều nhất tại từng bang/tỉnh theo từng ngày tính toán  $d$ . Với mỗi ngày  $d$ , cửa sổ thời gian chỉ xét tối đa 7 ngày trước đó, tức  $[d-7, d-1]$ , không bao gồm ngày hiện tại. Vì cửa sổ trượt từng ngày nên output có thể chứa các ngày không có order trực tiếp trong dữ liệu gốc, kể cả các ngày ngay sau ngày order cuối cùng nếu cửa sổ quá khứ vẫn còn dữ liệu đóng góp.

Một dòng dữ liệu được xem là “bought” khi thỏa đồng thời hai điều kiện: cột **Status** chứa chuỗi **shipped** sau khi chuẩn hóa chữ hoa/thường, và cột **Qty** là số nguyên dương. Cách hiểu này bao phủ các trạng thái như **Shipped**, **Shipped - Delivered to Buyer**, **Shipped - Returned to Seller**, đúng với mô tả trong đề bài là status có chứa “shipped”.

Đơn vị đo “mua nhiều nhất” trong lời giải là tổng **Qty** theo size trong cửa sổ, không chỉ là số dòng

order. Với mỗi khóa (`ship-state`, `date`), chương trình so sánh tổng quantity của từng size và chọn size có tổng lớn nhất. Nếu nhiều size có cùng tổng quantity, chương trình phá hòa bằng thứ tự từ điển tăng dần của chuỗi size để kết quả ổn định và tái lập được.

Output của task là một CSV thường trên filesystem, có header và đúng tên `Task_1-1.csv`. Schema thực tế:

| Cột                           | Kiểu dữ liệu | Ý nghĩa                                     |
|-------------------------------|--------------|---------------------------------------------|
| <code>ship-state</code>       | String       | Bang/tỉnh cần tính toán                     |
| <code>date</code>             | String       | Ngày tính toán $d$ , định dạng MM-dd-yy     |
| <code>most_bought_size</code> | String       | Size có tổng quantity cao nhất trong window |
| <code>max_quantity</code>     | Integer      | Tổng quantity của size thắng trong window   |

### 3.2 Phân rã thành các bước cơ bản (Decomposition)

Lời giải được triển khai theo tư duy MapReduce trên Spark RDD, gồm các bước sau:

1. **Đọc và kiểm tra schema input.** Chương trình đọc CSV, parse header và kiểm tra đủ các cột bắt buộc: `Date`, `Status`, `Qty`, `Size`, `ship-state`. Nếu thiếu cột, chương trình dừng với thông báo lỗi rõ ràng.
2. **Parse và chuẩn hóa bản ghi.** Mỗi dòng được parse bằng parser CSV tự viết để xử lý dấu phẩy trong quoted fields. Ngày được parse theo các format phổ biến như MM-dd-yy, M-d-yy, MM/dd/yy, M/d/yy; `Qty` được ép sang số nguyên dương; `ship-state` được trim và chuẩn hóa viết hoa để tránh tách cùng một bang thành nhiều key.
3. **Lọc item được mua.** Chỉ giữ các dòng có `Status` chứa `shipped`, `Qty`  $> 0$ , đồng thời `ship-state` và `Size` không rỗng. Bước này giảm dữ liệu trước khi phát sinh khóa window.
4. **Sinh đóng góp sliding window.** Với một purchase tại ngày  $t$ , chương trình emit đóng góp cho các ngày tính toán  $d = t + 1, t + 2, \dots, t + 7$ . Điều này tương đương với việc purchase tại ngày  $t$  nằm trong window quá khứ của các ngày  $d$  đó.
5. **Cộng tổng quantity theo (state, day, size).** Các đóng góp cùng bang, cùng ngày tính toán và cùng size được cộng bằng `reduceByKey`.

6. **Chọn size thắng theo (state, day).** Kết quả ở bước trước được đổi khóa thành (state, day), sau đó reducer chọn size có tổng quantity lớn nhất; nếu hòa thì chọn size có chuỗi nhỏ hơn theo thứ tự từ điển.
7. **Sắp xếp và ghi single CSV.** Kết quả được sort theo state và day, thêm header, coalesce(1) rồi ghi ra thư mục tạm. Sau đó chương trình rename part file thành một file CSV duy nhất.

### 3.3 Lý do phân rã

Phần khó nhất của bài toán là sliding window. Nếu tính trực tiếp từng ngày  $d$  rồi quét ngược lại 7 ngày, chi phí sẽ lặp lại nhiều lần. Lời giải chọn hướng ngược lại: một order tại ngày  $t$  tự phát sinh đóng góp cho tối đa 7 ngày tương lai mà nó có thể thuộc về window. Nhờ vậy, sliding window được chuyển thành bài toán gom nhóm key-value thông thường.

Cách phân rã này phù hợp với MapReduce vì mỗi bước đều có key rõ ràng. Bước map sinh key (state, day, size); bước reduce đầu cộng quantity; bước reduce thứ hai chọn winner cho từng (state, day). Việc lọc dữ liệu hợp lệ trước khi phát sinh 7 đóng góp cũng giúp giảm số bản ghi trung gian và giảm chi phí shuffle.

### 3.4 Chiến lược thực thi & Triển khai mã nguồn

Task 1.1 được cài đặt trong file `src/Task_1-1/source/Task1_1.scala`. Chương trình nhận hai tham số: đường dẫn CSV đầu vào và đường dẫn file CSV đầu ra. Logic chính dùng Spark RDD transformations như `mapPartitions`, `flatMap`, `reduceByKey`, `sortBy`, `saveAsTextFile`; không dùng DataFrame query hoặc Spark SQL cho phần tính toán.

**Lệnh chạy mẫu:**

```
1 cat > /tmp/task1_1.runner.scala <<'EOF'
2 :load src/Task_1-1/source/Task1_1.scala
3 Task1_1.main(Array("data/input/Amazon Sale Report.csv",
4   "data/output/Task_1-1.csv"))
5 sys.exit(0)
6 EOF
7 SPARK_LOCAL_IP=127.0.0.1 SPARK_LOCAL_HOSTNAME=localhost \
8   spark-shell < /tmp/task1_1.runner.scala
```

**Parse và lọc purchase hợp lệ.**

```
1 val purchases = parsedRows.flatMap { case (epochDay, state, size, status,
    qtyText) =>
2   parsePositiveInt(qtyText).filter(_ => status.contains("shipped")).flatMap {
    qty =>
3     if (state.nonEmpty && size.nonEmpty) Some(Purchase(epochDay, state, size,
        qty))
4     else None
5   }
6 }
```

Sinh đóng góp cho sliding window.

```
1 val windowContributions = purchases.flatMap { purchase =>
2   (1L to 7L).iterator.flatMap { offset =>
3     val targetDay = purchase.orderEpochDay + offset
4     if (targetDay >= minEpochDay) {
5       Some(((purchase.state, targetDay, purchase.size), purchase.qty))
6     } else {
7       None
8     }
9   }
10 }
```

Cộng tổng quantity và chọn size thắng.

```
1 val sizeTotals = windowContributions.reduceByKey(_ + _)
2
3 val winners = sizeTotals
4   .map { case ((state, day, size), totalQty) => ((state, day), (size, totalQty))
5     }
6   .reduceByKey { (left, right) =>
7     val (leftSize, leftQty) = left
8     val (rightSize, rightQty) = right
9     if (leftQty > rightQty) left
10    else if (rightQty > leftQty) right
11    else if (leftSize <= rightSize) left
12    else right
13  }
```

### 3.5 Phân tích chuyên sâu

#### 3.5.1 Pseudocode tổng quan

---

**Algorithm 1** Sliding Window — Most Bought Size per State (MapReduce)

---

**Require:** Dataset  $\mathcal{D}$  with columns (Date, Status, Qty, Size, ship-state)

**Ensure:** CSV file: (State, Date  $d$ , MostBoughtSize, MaxQty)

1:  $d_{\min} \leftarrow \min \text{ epoch day in } \mathcal{D}$

— MAP PHASE —

2: **for all** row  $r \in \mathcal{D}$  where Status contains “shipped”  $\wedge$  Qty  $> 0$  **do**

3:      $t \leftarrow \text{epochDay}(r.\text{Date})$

4:     **for**  $\delta = 1$  **to** 7 **do**

5:         **if**  $d_{\min} \leq t + \delta$  **then**

6:             **emit**  $\langle (r.\text{State}, t + \delta, r.\text{Size}), r.\text{Qty} \rangle$

7:         **end if**

8:     **end for**

9: **end for**

— REDUCE PHASE 1 —

10: **for all** key (State, Day, Size) **do**

11:      $\text{totalQty} \leftarrow \sum \text{ values}$

▷ `reduceByKey(_ + _)`

12: **end for**

— REDUCE PHASE 2 —

13: **for all** key (State, Day) **do**

14:      $\text{winner} \leftarrow \arg \max_{\text{Size}} \text{ totalQty}$

▷ tie-break: lexicographic

15:     **emit** (State, Day, winner.Size, winner.Qty)

16: **end for**

---

Phạm vi của sổ trượt cho ngày tính toán  $d$  được định nghĩa:

$$W(d) = [d - 7, d - 1]$$

Tương đương, một purchase tại ngày  $t$  đóng góp cho các ngày tính toán  $d \in [t + 1, t + 7]$ .

#### 3.5.2 Tại sao naive nested-loop bất khả thi

Cách tiếp cận trực tiếp là: với mỗi ngày  $d$  và mỗi state  $s$ , quét toàn bộ dataset để lọc các dòng có ngày thuộc  $[d - 7, d - 1]$  và cùng state, rồi đếm quantity theo size. Gọi  $D$  là số ngày distinct,  $S$  là

số state,  $N$  là tổng số dòng; phương pháp này có chi phí  $O(D \times S \times N)$ . Với  $D \approx 120$ ,  $S \approx 36$ ,  $N \approx 128\,000$ , ta có  $\sim 5.5 \times 10^8$  phép so sánh — quá tốn kém và không tận dụng được tính song song của MapReduce.

### 3.5.3 Key design giải quyết vấn đề thế nào

Thay vì “kéo ngược” từ ngày  $d$  ra quá khứ, lời giải “đẩy xuôi” từ mỗi purchase: một order tại ngày  $t$  tự phát sinh 7 bản ghi đóng góp cho các ngày  $t + 1$  đến  $t + 7$ . Key emit là `(state, targetDay, size)`, biến sliding window thành bài toán gom nhóm key-value:

- **Reduce lần 1** (`reduceByKey(_ + _)`): cộng tổng quantity cho mỗi `(state, day, size)`. Chi phí:  $O(N \times 7)$  bản ghi emit, sau đó shuffle và reduce — hoàn toàn parallelizable.
- **Reduce lần 2**: đổi key thành `(state, day)`, chọn size có tổng quantity lớn nhất. Nếu hòa, phá hòa bằng thứ tự từ điển tăng dần.

Cách này đạt chi phí  $O(7N)$  cho map phase và  $O(K \log K)$  cho sort/reduce (với  $K$  là số key distinct), tốt hơn naive approach nhiều bậc.

### 3.5.4 Xử lý unseen timestamps trong output

Vì window trượt mỗi ngày, kết quả có thể chứa ngày  $d$  mà *không có order nào* xảy ra đúng ngày  $d$  trong dataset gốc. Ví dụ: nếu không có order ngày 04-05-22 nhưng có orders từ 03-29-22 đến 04-04-22, thì ngày 04-05-22 vẫn xuất hiện trong output vì window  $[03 - 29, 04 - 04]$  có dữ liệu.

Cơ chế emission tự nhiên giải quyết điều này: khi một purchase ở ngày  $t$  emit đóng góp cho  $t + 1$  đến  $t + 7$ , các ngày “trống” trong khoảng đó vẫn nhận được bản ghi. Code không chặn ở ngày lớn nhất của dataset, vì đề cho phép unseen timestamp xuất hiện; với dữ liệu gốc có ngày cuối là 06-29-22, output hợp lệ có thể kéo dài đến 07-06-22 do các order cuối vẫn nằm trong cửa sổ quá khứ của những ngày này.

## 3.6 Kết quả thực thi & Xuất dữ liệu

File output đã được tạo tại `data/output/Task_1-1.csv`. File có 3,544 dòng bao gồm header, tương ứng 3,543 dòng kết quả. Một số dòng đầu của output:

```

1 ship-state,date,most_bought_size,max_quantity
2 ANDAMAN & NICOBAR,04-02-22,M,1
3 ANDAMAN & NICOBAR,04-03-22,S,2
4 ANDAMAN & NICOBAR,04-04-22,S,3
5 ANDAMAN & NICOBAR,04-05-22,S,3
6 ANDAMAN & NICOBAR,04-06-22,S,3
7 ANDAMAN & NICOBAR,04-07-22,S,4
8 ANDAMAN & NICOBAR,04-08-22,S,4
9 ANDAMAN & NICOBAR,04-09-22,XL,5

```

**Kiểm chứng thủ công.** Với ship-state = ANDAMAN & NICOBAR và ngày tính toán 04-09-22, cửa sổ quá khứ là từ 04-02-22 đến 04-08-22. Sau khi lọc các dòng có status chứa shipped và Qty > 0, tổng quantity theo size trong window là:

| Size | Tổng Qty trong window |
|------|-----------------------|
| XL   | 5                     |
| L    | 4                     |
| S    | 4                     |
| XS   | 4                     |
| XXL  | 3                     |
| 3XL  | 1                     |
| M    | 1                     |

Size XL có tổng quantity cao nhất là 5, khớp với dòng output ANDAMAN & NICOBAR,04-09-22,XL,5. Kiểm chứng này cho thấy cách phát sinh đóng góp  $t+1$  đến  $t+7$  tương đương với cửa sổ  $[d-7, d-1]$  của đề bài.

### 3.7 Minh chứng bằng hình ảnh

Các hình dưới đây là minh chứng chạy chương trình, preview output và kiểm chứng thủ công cho Task 1.1.

```

scala> Task1_1.main(Array("data/input/Amazon Sale Report.csv", "data/output/Task_1-1.csv"))
26/05/19 15:28:06 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
Task 1-1 completed. Output written to data/output/Task_1-1.csv

scala> sys.exit(0)

```

Hình 1: Kết quả chạy chương trình Task 1.1

| 1  | ship-state   | date     | most_bought_size | max_quantity |
|----|--------------|----------|------------------|--------------|
| 2  | ANDAMAN & NI | 04-02-22 | M                | 1            |
| 3  | ANDAMAN & NI | 04-03-22 | S                | 2            |
| 4  | ANDAMAN & NI | 04-04-22 | S                | 3            |
| 5  | ANDAMAN & NI | 04-05-22 | S                | 3            |
| 6  | ANDAMAN & NI | 04-06-22 | S                | 3            |
| 7  | ANDAMAN & NI | 04-07-22 | S                | 4            |
| 8  | ANDAMAN & NI | 04-08-22 | S                | 4            |
| 9  | ANDAMAN & NI | 04-09-22 | XL               | 5            |
| 10 | ANDAMAN & NI | 04-10-22 | XS               | 5            |
| 11 | ANDAMAN & NI | 04-11-22 | XL               | 4            |
| 12 | ANDAMAN & NI | 04-12-22 | XL               | 4            |

Hình 2: Một phần nội dung file Task\_1-1.csv

```

May 19 6:40 PM • en 100%
pearspringmind@master:~/Studying/Big Data/Lab03/Lab03_BigData

[pearspringmind@master: Lab03_BigData]$ python check_1.1.py
=====
KIỂM CHỨNG THỦ CÔNG TASK 1.1: ANDAMAN & NICOBAR
Cửa sổ trượt (7 ngày trước: ngày 04-09-22): 02-04-22 -> 08-04-22
=====

[1] Danh sách các đơn hàng hợp lệ trong cửa sổ:
=====
date      | size | qty | status
-----
04-02-22  | S    | 1   | Shipped
04-02-22  | S    | 1   | Shipped
04-02-22  | XL   | 1   | Shipped
04-03-22  | L    | 1   | Shipped
04-03-22  | L    | 1   | Shipped
04-03-22  | S    | 1   | Shipped
04-03-22  | XS   | 1   | Shipped
04-03-22  | XS   | 1   | Shipped
04-03-22  | XXL  | 1   | Shipped
04-04-22  | XS   | 1   | Shipped
04-05-22  | XXL  | 1   | Shipped - Delivered to Buyer
04-06-22  | S    | 1   | Shipped
04-06-22  | XL   | 1   | Shipped - Picked up
04-06-22  | XL   | 1   | Shipped - Picked up
04-06-22  | XXL  | 1   | Shipped - Picked up
04-07-22  | M    | 1   | Shipped
04-07-22  | XL   | 1   | Shipped
04-08-22  | L    | 1   | Shipped - Delivered to Buyer
04-08-22  | L    | 1   | Shipped - Delivered to Buyer
04-08-22  | XL   | 1   | Shipped
04-08-22  | XS   | 1   | Shipped
04-08-22  | XXL  | 1   | Shipped

[2] Tổng hợp số lượng (qty) theo từng Size:
=====
size      | Tổng qty
-----
XL        | 5
L         | 4
S         | 4
XS        | 4
XXL       | 3
3XL       | 1
M         | 1

[3] Kết luận phần tử thắng cuộc (Most Bought Size):
-> Size bán chạy nhất: XL
-> Tổng số lượng (Max qty): 5
-> Kết quả nhập với output: ANDAMAN & NICOBAR,04-09-22,XL,5
=====
[pearspringmind@master: Lab03_BigData]$

```

Hình 3: Kiểm chứng thủ công sliding window cho một state-date mẫu

## 4 Task 1.2: State-level Median Variety per Month

### 4.1 Phân tích truy vấn (Query Analysis)

Bài toán yêu cầu tính **median variety** cấp bang cho mỗi tháng. Cụ thể, với mỗi cặp (**ship-state**, **month**), ta cần xác định giá trị trung vị (median) của “variety” trên tập các style thỏa điều kiện đã phục vụ ít nhất một size từ XXL trở lên.

**Định nghĩa variety.** Variety của một style trong phạm vi (**state**, **month**) là số lượng SKU phân biệt (distinct) gắn với style đó trong cùng phạm vi. Variety *không phải* số order hay số size, mà là số mã sản phẩm duy nhất đã xuất hiện cho style đó tại bang và tháng xét.

**Điều kiện qualifying style.** Một style được xem là qualify (hợp lệ) nếu trong cùng phạm vi (**state**, **month**, **style**) tồn tại *ít nhất một* record có cột **Size** thuộc nhóm từ XXL trở lên. Các size code được hỗ trợ bao gồm: XXL, 2XL, XXXL, 3XL, 4XL, 5XL, 6XL. Ngoài ra, hàm kiểm tra trong code còn sử dụng biểu thức chính quy  $([2-9]|\d{2,})XL$  để bao phủ các size lớn hơn nữa (ví dụ 7XL, 10XL).

**Điểm quan trọng.** Sau khi một style được xác nhận là qualify, variety của style đó vẫn phải đếm *tất cả* các SKU distinct trong phạm vi (**state**, **month**), kể cả các SKU có size nhỏ hơn XXL. Điều kiện  $\text{size} \geq \text{XXL}$  chỉ dùng để quyết định style có được đưa vào tập tính median hay không, không dùng để lọc SKU.

**Cách tính median.** Sau khi thu được danh sách variety của tất cả qualifying styles trong một (**state**, **month**), danh sách này được sắp xếp tăng dần. Nếu số phần tử lẻ, median là phần tử ở vị trí giữa; nếu chẵn, median là trung bình cộng của hai phần tử giữa. Kết quả được làm tròn đến đúng 4 chữ số thập phân.

**Schema output.** File kết quả **Task\_1-2.csv** có 3 cột:

| Cột            | Kiểu dữ liệu    | Ý nghĩa                                  |
|----------------|-----------------|------------------------------------------|
| ship-state     | String          | Bang/tỉnh đã chuẩn hóa viết hoa          |
| month          | String          | Tháng, định dạng YYYY-MM                 |
| median_variety | Double (4 d.p.) | Median variety của các qualifying styles |

### 4.2 Phân rã thành các bước cơ bản (Decomposition)

Lời giải được triển khai theo tư duy MapReduce trên Spark RDD, gồm các bước sau:

1. **Đọc input và chọn cột.** Chương trình đọc file CSV bằng `sc.textFile` trên đường dẫn `file://` tuyệt đối, parse header bằng parser CSV tự viết và kiểm tra đủ 5 cột cần thiết: `Date`, `SKU`, `Style`, `Size`, `ship-state`. Các dòng thiếu một trong 5 trường này bị bỏ qua. Cột `ship-state` và `Size` được trim khoảng trắng và chuẩn hóa viết hoa. Cột `Date` được parse từ các định dạng `MM-dd-yy`, `M-d-yy`, `MM/dd/yy`, `M/d/yy` sang `YYYY-MM`; các dòng có ngày không hợp lệ bị bỏ qua.
2. **Map sang khóa style.** Mỗi record hợp lệ được emit thành cặp key-value với key là `(state, month, style)` và value là `(SKU, isAtLeastXXL)`. Đây là bước Map đầu tiên, phát tán dữ liệu theo khóa composite 3 thành phần để chuẩn bị cho bước gom nhóm tiếp theo.
3. **Reduce lần 1 – tính variety và cờ XXL.** Sử dụng `aggregateByKey` với accumulator là một `mutable.Set[String]` (chứa các SKU distinct) và một cờ boolean `hasXXL`. Hàm `seqOp` thêm SKU vào set và OR giá trị `isAtLeastXXL` vào cờ. Hàm `combOp` hợp nhất hai set và OR hai cờ lại với nhau. Kết quả: mỗi key `(state, month, style)` cho ra `(variety = |set|, hasXXL)`.
4. **Lọc style qualify.** Chỉ giữ lại các style có `hasXXL = true`. Bước filter này được thực hiện trước khi đổi khóa để giảm lượng dữ liệu cần shuffle trong bước tiếp theo.
5. **Map sang khóa state-month.** Đổi key từ `(state, month, style)` thành `(state, month)`, value là `variety` (số nguyên). Thành phần `style` trong key bị loại bỏ vì từ bước này trở đi ta chỉ cần gom variety theo bang và tháng.
6. **Reduce lần 2 – tính median.** Dùng `groupByKey` để gom tất cả variety của các qualifying styles cùng `(state, month)` thành một danh sách. Danh sách được sort tăng dần rồi truyền vào hàm `computeMedian` để tính giá trị trung vị, kết quả được format thành chuỗi 4 chữ số thập phân.
7. **Xuất CSV.** Kết quả được `sortBy` theo `(state, month)` rồi `collect()` về driver. Chương trình dùng `PrintWriter` của Java I/O để ghi header và từng dòng kết quả ra một file CSV duy nhất, tránh tạo thư mục part files đặc trưng của Hadoop.

### 4.3 Lý do phân rã

Bài toán đòi hỏi hai tầng aggregation vì hai lý do chính. Thứ nhất, cùng một style có thể chứa cả SKU size nhỏ (S, M, L) lẫn SKU size lớn (XXL, 3XL). Nếu lọc chỉ giữ các dòng có size  $\geq$  XXL trước khi đếm variety, ta sẽ mất các SKU size nhỏ và sai định nghĩa variety. Do đó, reducer đầu tiên phải *đồng thời* giữ toàn bộ tập SKU distinct và ghi nhận style đã từng phục vụ size từ XXL trở lên.

Thứ hai, bài toán yêu cầu tính median – một phép toán toàn cục trên danh sách đã sort, không thể thực hiện bằng một phép reduce đơn giản kiểu cộng dồn. Vì vậy cần tầng aggregation thứ hai: gom tất cả variety theo (state, month), sort danh sách rồi trích giá trị trung vị. Cách phân rã thành hai tầng reducer phù hợp với mô hình key-value processing vì mỗi reducer chỉ cần xử lý dữ liệu cùng key mà không phụ thuộc vào các key khác.

### 4.4 Chiến lược thực thi & Triển khai mã nguồn

Task 1.2 được cài đặt trong file `src/Task_1-2/source/Task1_2.scala`. Chương trình nhận hai tham số: đường dẫn CSV đầu vào và đường dẫn file CSV đầu ra. Toàn bộ phần xử lý chính dùng Spark RDD/key-value transformations, gồm `textFile`, `flatMap`, `aggregateByKey`, `filter`, `map`, `groupByKey`, `sortBy`; không dùng DataFrame query hoặc Spark SQL cho logic bài toán MapReduce.

Lệnh chạy mẫu:

```
1 cat > /tmp/task1_2.runner.scala <<'EOF'
2 :load src/Task_1-2/source/Task1_2.scala
3 Task1_2.main(Array("data/input/Amazon Sale Report.csv",
4   "data/output/Task1_2.csv"))
5 sys.exit(0)
6 EOF
7 spark-shell < /tmp/task1_2.runner.scala
```

Xác định size threshold.

```
1 def isAtLeastXXL(size: String): Boolean = {
2   val s = size.trim.toUpperCase
3   val xlSizes = Set("XXL", "2XL", "XXXL", "3XL",
4     "4XL", "5XL", "6XL")
5   xlSizes.contains(s) || s.matches("[2-9]\\d{2,}XL")
6 }
```

Hàm `isAtLeastXXL` trước tiên kiểm tra size có thuộc tập các giá trị phổ biến hay không. Nếu không, biểu thức chính quy sẽ bắt các size dạng `nXL` với  $n \geq 2$  hoặc số có từ hai chữ số trở lên, đảm bảo bao phủ mọi size từ `XXL` trở lên mà không nhận nhầm các giá trị như `XL` hay `1XL`.

### Map và Reduce lần 1 – tính variety per style.

```
1 val mapped = dataLines.flatMap(line => {
2   val fields = parseCsvLine(line)
3   val dateStr = field(fields, col.date).trim
4   val sku     = field(fields, col.sku).trim
5   val style   = field(fields, col.style).trim
6   val size    = field(fields, col.size).trim.toUpperCase
7   val state   = field(fields, col.state).trim.toUpperCase
8   if (dateStr.isEmpty || sku.isEmpty || style.isEmpty ||
9       size.isEmpty || state.isEmpty) None
10  else parseMonth(dateStr).map(month =>
11    ((state, month, style), (sku, isAtLeastXXL(size))))
12 })
13
14 val styleVariety = mapped.aggregateByKey((Set.empty[String], false))(
15   (acc, value) => {
16     (acc._1 + value._1, acc._2 || value._2)
17   },
18   (acc1, acc2) => {
19     (acc1._1 ++ acc2._1, acc1._2 || acc2._2)
20   }
21 ).map {
22   case ((state, month, _), (skuSet, hasXXL)) =>
23     ((state, month), (skuSet.size, hasXXL))
24 }
```

### Reduce lần 2 – lọc qualifying styles và tính median.

```
1 val medianVariety = styleVariety
2   .filter { case (_, (_, hasXXL)) => hasXXL }
3   .map { case (k, (size, _)) => (k, size) }
4   .groupByKey()
5   .flatMap { case ((state, month), sizesIter) =>
6     val varieties = sizesIter.toList.sorted
```

```
7     computeMedian(varieties).map(median =>
8         (state, month, median))
9 }
```

### Hàm tính median.

```
1 def computeMedian(sortedValues: List[Int]):
2     Option[String] = {
3     if (sortedValues.isEmpty) None
4     else {
5         val n = sortedValues.size
6         val median =
7             if (n % 2 == 1) sortedValues(n / 2).toDouble
8             else (sortedValues(n / 2 - 1) +
9                 sortedValues(n / 2)) / 2.0
10        Some(f"$median%.4f")
11    }
12 }
```

**Xử lý dữ liệu lỗi.** Các dòng có giá trị null ở bất kỳ cột nào trong 5 cột cần thiết đều bị loại bằng `.na.drop()` ngay từ đầu pipeline. Hàm `parseMonth` trả về `None` khi chuỗi ngày không đúng định dạng MM-dd-yy, và `flatMap` tự động bỏ qua các `None` này. Cột `ship-state` và `Size` được `trim` và chuyển `toUpperCase` để chuẩn hóa, tránh trường hợp cùng một bang nhưng bị viết khác kiểu chữ (ví dụ “Bihar” và “BIHAR”) dẫn đến trùng lặp key.

## 4.5 Phân tích chuyên sâu

### 4.5.1 Pseudocode tổng quan

---

**Algorithm 2** State-level Median Variety per Month (Two-pass MapReduce)
 

---

**Require:** Dataset  $\mathcal{D}$  with columns (Date, SKU, Style, Size, ship-state)

**Ensure:** CSV file: (State, Month, MedianVariety)

— **MAP** —

```

1: for all row  $r \in \mathcal{D}$  where all 5 columns  $\neq$  null do
2:   month  $\leftarrow$  parseMonth(r.Date)                                ▷ “YYYY-MM”
3:   emit  $\langle (r.State, month, r.Style), (r.SKU, isXXL(r.Size)) \rangle$ 
4: end for

```

— **REDUCE 1: variety per style** —

```

5: for all key (State, Month, Style) do
6:   skuSet  $\leftarrow$  collect distinct SKU from values
7:   hasXXL  $\leftarrow \bigvee$  isXXL flags from values
8:   variety  $\leftarrow |\text{skuSet}|$ 
9:   if hasXXL then
10:    emit  $\langle (State, Month), variety \rangle$ 
11:   end if
12: end for

```

— **REDUCE 2: median** —

```

13: for all key (State, Month) do
14:    $V \leftarrow$  sort(all variety values)
15:    $n \leftarrow |V|$ 
16:   if  $n$  is odd then
17:     median  $\leftarrow V[\lfloor n/2 \rfloor]$ 
18:   else
19:     median  $\leftarrow (V[n/2 - 1] + V[n/2]) / 2$ 
20:   end if
21:   emit (State, Month, median)
22: end for

```

---

Công thức tính median cho danh sách đã sắp xếp  $V = [v_1, v_2, \dots, v_n]$ :

$$\text{median}(V) = \begin{cases} v_{\lceil n/2 \rceil} & \text{nếu } n \text{ lẻ} \\ \frac{v_{n/2} + v_{n/2+1}}{2} & \text{nếu } n \text{ chẵn} \end{cases}$$

#### 4.5.2 Tại sao median KHÔNG thể tính bằng Combiner đơn giản

Trong MapReduce, Combiner cho phép giảm dữ liệu trước shuffle bằng cách thực hiện reduce cục bộ tại mỗi mapper. Tuy nhiên, Combiner chỉ hoạt động với các phép toán *associative* và *commutative* như sum, min, max. Median **không** thỏa tính chất này: trung vị của hai tập con không bằng trung vị của tập gộp.

**Ví dụ:** Partition A có variety  $[1, 3, 5]$ , median = 3. Partition B có variety  $[2, 4]$ , median = 3. Nhưng tập gộp  $[1, 2, 3, 4, 5]$  có median = 3 — trùng trong ví dụ này nhưng không đảm bảo trong trường hợp tổng quát (ví dụ  $A = [1, 10]$ ,  $B = [2, 3]$ : median A = 5.5, median B = 2.5, nhưng median gộp = 2.5).

Do đó, bước Reduce 2 phải dùng `groupByKey` (không phải `reduceByKey`) để gom toàn bộ danh sách variety về cùng reducer, sort rồi trích vị trí giữa. Đây là chi phí bắt buộc: shuffle toàn bộ variety values.

#### 4.5.3 Chiến lược tính median trong MapReduce context

Lời giải áp dụng chiến lược **two-pass aggregation**:

1. **Pass 1** (`aggregateByKey`): Gom theo (`state`, `month`, `style`) để tính variety =  $|\text{distinct SKU}|$  và cờ `hasXXL`. `aggregateByKey` cho phép dùng accumulator `mutable.Set` — hiệu quả hơn `groupByKey` vì chỉ cần duy trì set thay vì gom toàn bộ list.
2. **Pass 2** (`groupByKey`): Gom theo (`state`, `month`) để thu tất cả variety, sort rồi tính median. Ở pass này, `groupByKey` là bắt buộc vì median cần toàn bộ danh sách đã sort.

Lý do không dùng `TreeMap/SortedMap` trong reducer: trong Spark RDD context, `aggregateByKey` với `SortedMap` vẫn phải merge các map từ các partition — chi phí merge  $O(k \log k)$  tương đương sort sau `groupByKey`, nhưng phức tạp hóa code và tăng bộ nhớ do phải duy trì cấu trúc cây trong accumulator.

#### 4.5.4 Cách xác định “size $\geq$ XXL” — parsing logic

Hàm `isAtLeastXXL` kiểm tra size theo hai bước:

1. **Exact match**: So khớp trực tiếp với tập  $\{\text{XXL}, \text{2XL}, \text{XXXL}, \text{3XL}, \text{4XL}, \text{5XL}, \text{6XL}\}$  sau khi `trim.toUpperCase`.

2. **Regex fallback:** Pattern `([2-9]|\d{2,})XL` bắt các size dạng  $nXL$  với  $n \geq 2$  hoặc  $n$  có từ 2 chữ số trở lên (ví dụ 10XL, 12XL).

Thứ tự quan trọng: kiểm tra Set trước ( $O(1)$  lookup) vì đa số cases rơi vào đây; regex chỉ là fallback cho các size hiếm gặp. Lưu ý XL (chỉ 1 chữ X) và 1XL đều **không** qualify vì không khớp Set và regex yêu cầu  $n \geq 2$ .

## 4.6 Kết quả thực thi & Xuất dữ liệu

File output đã được tạo tại `data/output/Task_1-2.csv`. Thông tin chi tiết:

- Đường dẫn output: `data/output/Task_1-2.csv`.
- Số dòng bao gồm header: 129.
- Số dòng kết quả không tính header: 128.
- Kích thước file: 3,270 bytes.

Một số dòng đầu của output:

```
1 ship-state,month,median_variety
2 ANDAMAN & NICOBAR,2022-04,1.0000
3 ANDAMAN & NICOBAR,2022-05,1.0000
4 ANDAMAN & NICOBAR,2022-06,1.0000
5 ANDHRA PRADESH,2022-03,1.0000
6 ANDHRA PRADESH,2022-04,2.0000
7 ANDHRA PRADESH,2022-05,2.0000
8 ANDHRA PRADESH,2022-06,2.0000
9 ARUNACHAL PRADESH,2022-04,1.0000
10 ASSAM,2022-04,2.0000
```

**Kiểm chứng thủ công.** Chọn trường hợp `ship-state = CHANDIGARH`, `month = 2022-03` làm mẫu kiểm tra. Sau khi lọc trực tiếp trên file dữ liệu gốc `Amazon Sale Report.csv`, ta thu được đúng 1 record:

| Date     | Style   | SKU             | Size |
|----------|---------|-----------------|------|
| 03-31-22 | JNE3758 | JNE3758-KR-XXXL | 3XL  |

Phân tích: chỉ có 1 style (JNE3758) với 1 SKU distinct, do đó variety = 1. Style này có size 3XL  $\geq$  XXL nên qualify. Danh sách variety hợp lệ: [1]. Median của tập [1] bằng 1.0000. Kết quả này khớp hoàn toàn với dòng tương ứng trong file output: CHANDIGARH,2022-03,1.0000, xác nhận logic MapReduce hoạt động chính xác.

## 4.7 Minh chứng bằng hình ảnh

Các hình dưới đây là minh chứng chạy chương trình, preview output và kiểm chứng thủ công cho Task 1.2.

```
scala> Task1_2.main(Array("data/input/Amazon Sale Report.csv", "data/output/Task_1-2.csv"))
26/05/13 17:42:05 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
Task 1-2 hoàn thành: 128 dòng -> data/output/Task_1-2.csv
```

Hình 4: Kết quả chạy chương trình Task 1.2

| ship-state        | month   | median_variety |
|-------------------|---------|----------------|
| ANDAMAN & NICOBAR | 2022-04 | 1              |
| ANDAMAN & NICOBAR | 2022-05 | 1              |
| ANDAMAN & NICOBAR | 2022-06 | 1              |
| ANDHRA PRADESH    | 2022-03 | 1              |
| ANDHRA PRADESH    | 2022-04 | 2              |
| ANDHRA PRADESH    | 2022-05 | 2              |
| ANDHRA PRADESH    | 2022-06 | 2              |
| ARUNACHAL PRADESH | 2022-04 | 1              |
| ARUNACHAL PRADESH | 2022-05 | 1              |
| ARUNACHAL PRADESH | 2022-06 | 1              |
| ASSAM             | 2022-04 | 2              |
| ASSAM             | 2022-05 | 1.5            |
| ASSAM             | 2022-06 | 1              |
| BIHAR             | 2022-04 | 2              |
| BIHAR             | 2022-05 | 2              |
| BIHAR             | 2022-06 | 1              |
| CHANDIGARH        | 2022-03 | 1              |
| CHANDIGARH        | 2022-04 | 1              |

Hình 5: Một phần nội dung file Task\_1-2.csv

```

=====
KIỂM CHỨNG THỦ CÔNG: CHANDIGARH, Tháng 03-2022
=====

[1] Bảng dữ liệu gốc sau khi lọc (Filter):
      Date      Style      SKU Size
03-31-22 JNE3758 JNE3758-KR-XXXL 3XL

[2] Phân tích từng Style:
-> Style: JNE3758
  + Danh sách SKUs (1 SKUs): ['JNE3758-KR-XXXL']
  + Danh sách Sizes: ['3XL']
  => HỢP LỆ (Có size >= XXL)

[3] Kết quả cuối cùng:
-> Tập các variety hợp lệ để tính Median: [1]
-> MEDIAN VARIETY tính bằng tay: 1.0000
=====

```

Hình 6: Kiểm chứng thủ công median variety cho một state-month mẫu

## 5 Task 2.1: Percentage of Cancelled Orders per City

### 5.1 Phân tích truy vấn (Query Analysis)

Bài toán yêu cầu tính **phần trăm đơn hàng bị huỷ** (cancelled) theo từng thành phố (ship-city), trong đó chỉ xét các đơn hàng thỏa **đồng thời ba điều kiện**:

**Điều kiện 1 — Service level.** Đơn hàng phải có ship-service-level bằng "Standard". Trong code, phép so sánh được chuẩn hóa bằng `lower(trim(col("ship-service-level")))` == "standard" để tránh sai lệch do khoảng trắng thừa hoặc khác biệt chữ hoa/chữ thường.

**Điều kiện 2 — Ít nhất 3 temporally-valid promotions.** Mỗi đơn hàng chứa cột promotion-ids, là chuỗi các mã khuyến mãi phân tách bằng dấu phẩy. Một promotion được gọi là *temporally valid* (hợp lệ về mặt thời gian) nếu khoảng thời gian hoạt động của nó trên **toàn bộ dataset** kéo dài ít nhất 2 ngày. Cụ thể, với mỗi promotion id duy nhất xuất hiện trong dataset:

- Xác định ngày xuất hiện **đầu tiên** ( $d_{\min}$ ) và ngày xuất hiện **cuối cùng** ( $d_{\max}$ ).
- Tính khoảng hoạt động:  $\text{active\_days} = d_{\max} - d_{\min}$  (tính bằng số ngày).

- Promotion hợp lệ khi  $\text{active\_days} \geq 2$ .

Một đơn hàng thỏa điều kiện này nếu trong danh sách `promotion-ids` của nó có ít nhất 3 promotion hợp lệ, kể cả các promotion do Amazon phát hành.

**Điều kiện 3 — Amount thấp hơn trung bình bang.** Tính giá trị trung bình Amount theo từng `ship-state` trên tập con các đơn hàng có `Fulfilment = "Merchant"` và `Courier Status = "Shipped"`. Đơn hàng đang xét chỉ được giữ nếu:

$$\text{Amount}_{\text{order}} < \overline{\text{Amount}}_{\text{Merchant, Shipped}}^{(\text{same state})}$$

**Tính phần trăm cancelled.** Sau khi lọc ra tất cả đơn hàng thỏa đồng thời ba điều kiện, nhóm theo `ship-city` đã chuẩn hóa:

$$\text{cancelled\_percentage} = \frac{|\{o : \text{Status}(o) = \text{"Cancelled"}\}|}{|\text{tổng đơn qualified}|} \times 100$$

Code xác định trạng thái cancelled bằng `lower(trim(col("Status"))) === "cancelled"`, tức so khớp chính xác sau chuẩn hóa.

**Schema output Parquet:**

| Cột                               | Kiểu   | Ý nghĩa                                    |
|-----------------------------------|--------|--------------------------------------------|
| <code>ship-city</code>            | String | Thành phố giao hàng đã chuẩn hóa viết hoa  |
| <code>cancelled_percentage</code> | Double | Tỷ lệ huỷ (%), làm tròn 4 chữ số thập phân |

## 5.2 Phân rã thành các bước cơ bản (Decomposition)

Lời giải được phân rã thành 9 bước tuần tự, mỗi bước tương ứng một DataFrame transformation rõ ràng:

1. **Đọc CSV và cast kiểu dữ liệu (df).** Đọc file CSV với header, tắt `inferSchema` để kiểm soát kiểu thủ công. Parse cột `Date` sang kiểu `date` bằng format `MM-dd-yy`, cast `Amount` sang `DoubleType`, chuẩn hóa `ship-city/ship-state` bằng `upper(trim(...))`, và tạo cột `order_row_id` bằng `monotonically_increasing_id()` để định danh duy nhất từng dòng sau các phép join. DataFrame này được `cache()` vì sử dụng lại ở nhiều nhánh.

2. **Explode promotion-ids thành từng dòng riêng** (promoExploded). Lọc bỏ các dòng có promotion-ids null hoặc rỗng. Dùng `split` tách chuỗi theo dấu phẩy, `transform + trim` để loại khoảng trắng thừa, rồi `explode` thành từng promotion id riêng lẻ. Bước này cần thiết vì active period là thuộc tính *toàn cục* của promotion id — phải gom tất cả ngày xuất hiện trên toàn dataset.
3. **Tính promotion validity** (promoValidity). GroupBy `promo_id`, tính `min(parsed_date)` và `max(parsed_date)`, sau đó `datediff` để ra `active_days`. Lọc giữ lại các promotion có `active_days ≥ 2`.
4. **Đếm valid promotions theo từng order bằng DataFrame**. Từ df, chương trình `explode` promotion-ids theo `order_row_id`, join với bảng `promoValidity`, rồi `groupBy("order_row_id")` để đếm số promotion hợp lệ bằng `countDistinct`. Cách này không dùng UDF và không collect tập promotion về driver, nên toàn bộ logic vẫn hiển thị trong Catalyst plan.
5. **Áp dụng Điều kiện 1 và 2** (afterCond12). Filter `ship-service-level = "Standard"`, sau đó left join với bảng `validPromotionCounts` theo `order_row_id`; order không có promotion hợp lệ nhận 0. Cuối cùng filter `valid_promo_count ≥ 3`.
6. **Tính trung bình Amount theo state** (stateAvgDf). Từ df gốc (không phải afterCond12), lọc `Fulfilment = "Merchant"` và `Courier Status = "Shipped"`, loại Amount null, rồi `groupBy("ship_state_norm")` để tính trung bình. Kết quả là bảng nhỏ chứa ngưỡng trung bình cho mỗi state đã chuẩn hóa.
7. **Join và áp dụng Điều kiện 3** (qualifiedDf). Inner join afterCond12 với stateAvgDf trên cột `ship_state_norm`, rồi filter `Amount < merchant_shipped_avg`. Sau bước này, chỉ còn lại các đơn thỏa đồng thời cả ba điều kiện.
8. **Tính cancelled percentage theo city** (resultDf). GroupBy `ship_city_norm`, đếm tổng đơn và đếm đơn cancelled bằng `sum(when(... === "cancelled", 1).otherwise(0))`. Tính tỷ lệ phần trăm và làm tròn 4 chữ số.
9. **Xuất single Parquet file**. `coalesce(1)` để gộp về 1 partition, ghi vào thư mục tạm, rồi rename part file thành file Parquet đơn lẻ tại đường dẫn output. Xóa checksum sidecar file.

### 5.3 Lý do phân rã

**Temporally-valid promotion phải tính toàn cục trước.** Active period của một promotion id được xác định bởi khoảng cách giữa ngày xuất hiện đầu tiên và cuối cùng *trên toàn bộ dataset*, không phải chỉ trong một order riêng lẻ. Do đó, bước explode-groupBy-datediff phải xử lý toàn bộ dữ liệu trước khi kết quả có thể áp dụng ngược lại từng order. Đây là lý do pipeline chia thành hai nhánh song song: nhánh tính validity và nhánh xử lý order chính.

**Average Amount theo state phải tính riêng rồi join.** Điều kiện thứ ba yêu cầu so sánh Amount của mỗi order với trung bình Amount cùng state nhưng chỉ trên tập con Merchant+Shipped. Vì tập con này khác với tập order đang xét (Standard service level), ta không thể dùng window function trực tiếp mà phải tính aggregate riêng rồi join theo `ship_state_norm`. Bảng `stateAvgDf` nhỏ, nên Spark có thể chọn BroadcastHashJoin, tránh shuffle.

**Thứ tự áp dụng điều kiện tối ưu shuffle cost.** Code áp dụng Điều kiện 1 (filter Standard) trước khi join với bảng đếm promotion hợp lệ để giảm số dòng order chính tham gia join. Điều kiện 3 được áp dụng sau cùng vì cần kết quả join với `stateAvgDf`. Nếu đảo thứ tự — join state average trước rồi mới lọc service level/promotion — khối lượng dữ liệu tham gia join sẽ lớn hơn đáng kể.

### 5.4 Chiến lược thực thi & Triển khai mã nguồn

Task 2.1 được cài đặt trong file `src/Task_2-1/source/Task2_1.scala`, sử dụng **hoàn toàn DataFrame/Dataset API** — không dùng Spark SQL string query. Chương trình nhận hai tham số: đường dẫn CSV đầu vào và đường dẫn file Parquet đầu ra.

**Lệnh chạy mẫu:**

```
1 cat > /tmp/task2_1.runner.scala <<'EOF'
2 :load src/Task_2-1/source/Task2_1.scala
3 Task2_1.main(Array("data/input/Amazon Sale Report.csv",
4   "data/output/Task_2-1.parquet"))
5 sys.exit(0)
6 EOF
7 spark-shell < /tmp/task2_1.runner.scala
```

**Explode và tính promotion validity.**

```
1 val promoExploded = df
2   .filter(col("promotion-ids").isNotNull
```

```

3      && col("promotion-ids") != "")
4      .select(col("parsed_date"),
5              explode(transform(
6                  split(col("promotion-ids"), ","),
7                  (promoId: Column) => trim(promoId)
8              )).as("promo_id"))
9      .filter(col("promo_id") != "")
10
11 val promoValidity = promoExploded
12     .groupBy("promo_id")
13     .agg(min("parsed_date").as("first_date"),
14          max("parsed_date").as("last_date"))
15     .withColumn("active_days",
16                 datediff(col("last_date"), col("first_date")))
17     .filter(col("active_days") >= 2)
18     .select("promo_id")
    
```

Lý do dùng `transform + trim` trước `explode` thay vì `trim` sau: đảm bảo mỗi token đã sạch trước khi tạo dòng mới, tránh tạo dòng thừa với promotion id rỗng do trailing comma.

### Đếm valid promotions theo order bằng DataFrame.

```

1 val orderPromos = df
2   .filter(col("promotion-ids").isNotNull &&
3           trim(col("promotion-ids")) != "")
4   .select(col("order_row_id"),
5           explode(transform(split(col("promotion-ids"), ","),
6                               (promoId: Column) => trim(promoId))).as("promo_id"))
7   .filter(col("promo_id") != "")
8
9 val validPromotionCounts = orderPromos
10  .join(broadcast(promoValidity), Seq("promo_id"), "inner")
11  .groupBy("order_row_id")
12  .agg(countDistinct("promo_id").as("valid_promo_count"))
    
```

Chiến lược này giữ được tính declarative của Structured API. `promoValidity` chỉ có 185 promotion hợp lệ nên được Spark broadcast làm build side khi join với `orderPromos`; phần đếm theo `order_row_id` là aggregation rõ ràng, có thể nhìn thấy trong physical plan.

### Filter service level và đếm promotions.

```
1 val afterCond12 = df
2   .filter(lower(trim(
3     col("ship-service-level"))) === "standard")
4   .join(validPromotionCounts, Seq("order_row_id"), "left")
5   .withColumn("valid_promo_count",
6     coalesce(col("valid_promo_count"), lit(0L)))
7   .filter(col("valid_promo_count") >= 3)
```

Tính trung bình Amount theo state.

```
1 val stateAvgDf = df
2   .filter(
3     lower(trim(col("Fulfilment"))) === "merchant" &&
4     lower(trim(col("Courier Status"))) === "shipped")
5   .filter(col("Amount").isNotNull)
6   .groupBy("ship_state_norm")
7   .agg(avg("Amount").as("merchant_shipped_avg"))
```

Join điều kiện 3 và tính cancelled percentage.

```
1 val joinedDf = afterCond12.join(
2   stateAvgDf, Seq("ship_state_norm"), "inner")
3
4 val qualifiedDf = joinedDf
5   .filter(col("Amount").isNotNull)
6   .filter(col("Amount") < col("merchant_shipped_avg"))
7
8 val resultDf = qualifiedDf
9   .groupBy(col("ship_city_norm"))
10  .agg(
11    count("*").as("total_orders"),
12    sum(when(lower(trim(col("Status"))) === "cancelled",
13      1).otherwise(0)).as("cancelled_orders"))
14  .withColumn("cancelled_percentage",
15    round(col("cancelled_orders") /
16      col("total_orders") * 100.0, 4))
17  .select(col("ship_city_norm").as("ship-city"),
18    col("cancelled_percentage"))
19  .orderBy("ship-city")
```

Việc chuẩn hóa ship-city trước khi group là cần thiết vì dữ liệu có khác biệt chữ hoa/thường và khoảng trắng. Kết quả cuối cùng bám đúng câu chữ đề bài: tính percentage cho từng city.

## 5.5 Phân tích chuyên sâu — Kế hoạch thực thi (Execution Plan Analysis)

### 5.5.1 Output của explain(true)

Dưới đây là kế hoạch thực thi mở rộng (extended execution plan) thu được từ lệnh `resultDf.explain(true)` khi chạy chương trình:

```

1 == Physical Plan ==
2 AdaptiveSparkPlan isFinalPlan=false
3 +- Sort [ship-city ASC NULLS FIRST], true, 0
4   +- Exchange rangepartitioning(ship-city ASC NULLS FIRST, 200)
5     +- HashAggregate(keys=[ship_city_norm], functions=[sum(...), count(1)])
6       +- Exchange hashpartitioning(ship_city_norm, 200)
7         +- HashAggregate(keys=[ship_city_norm], functions=[partial_sum(...), partial_count(1)])
8           +- Project [Status, ship_city_norm]
9             +- BroadcastHashJoin [ship_state_norm], [ship_state_norm], Inner, BuildRight,
10              (Amount < merchant_shipped_avg), false
11              :- Project [Status, Amount, ship_city_norm, ship_state_norm]
12                : +- BroadcastHashJoin [order_row_id], [order_row_id], Inner, BuildRight, false
13                  :   :- Filter lower(trim(ship-service-level)) = standard
14                    :   +- BroadcastExchange HashedRelationBroadcastMode(order_row_id)
15                      :   +- Filter coalesce(valid_promo_count, 0) >= 3
16                        :   +- HashAggregate(keys=[order_row_id],
17                          functions=[count(distinct promo_id)])
18                          +- Exchange hashpartitioning(order_row_id, 200)
19                          +- HashAggregate(keys=[order_row_id, promo_id])
20                          +- Exchange hashpartitioning(order_row_id, promo_id, 200)
21                          +- BroadcastHashJoin [promo_id], [promo_id],
22                          Inner, BuildRight, false
23                          :- Generate explode(transform(split(promotion-ids, ","), trim))
24                          +- BroadcastExchange HashedRelationBroadcastMode(promo_id)
25                          +- HashAggregate(keys=[promo_id],
26                          functions=[min(parsed_date), max(parsed_date)])
27                          +- Exchange hashpartitioning(promo_id, 200)
28      +- BroadcastExchange HashedRelationBroadcastMode(ship_state_norm)
29        +- HashAggregate(keys=[ship_state_norm], functions=[avg(Amount)])
30          +- Exchange hashpartitioning(ship_state_norm, 200)

```

### 5.5.2 Phân tích Physical Join Strategy

Pipeline chứa một phép join chính: `afterCond12 ⋈ship_state_norm stateAvgDf`.

| Join                                   |   | Strategy thực tế  | Giải thích                                                                                             |
|----------------------------------------|---|-------------------|--------------------------------------------------------------------------------------------------------|
| orderPromos<br>promoValidity           | ⋈ | BroadcastHashJoin | promoValidity chỉ có 185 dòng nên được broadcast làm build side.                                       |
| standardOrders<br>validPromotionCounts | ⋈ | BroadcastHashJoin | Bảng order đủ điều kiện promotion sau aggregation nhỏ hơn ngưỡng broadcast, Spark chọn BuildRight.     |
| afterCond12<br>stateAvgDf              | ⋈ | BroadcastHashJoin | stateAvgDf chỉ có bảng trung bình theo state, nhỏ hơn nhiều so với broadcast threshold mặc định 10 MB. |

**Build side** của cả ba BroadcastHashJoin đều là phía phải trong physical plan: `promoValidity`, `validPromotionCounts`, và `stateAvgDf`. Spark serialize các bảng nhỏ này, broadcast đến executor, rồi hash lookup cục bộ trên partition của bảng lớn hơn. Nhờ vậy, các join chính không cần shuffle cả hai phía theo join key.

Nếu `stateAvgDf` vượt 10 MB (cấu hình `spark.sql.autoBroadcastJoinThreshold`), Spark sẽ chuyển sang SortMergeJoin — yêu cầu shuffle cả hai bảng theo join key — tốn kém hơn đáng kể. Trong trường hợp này, với chỉ ~36 dòng, BroadcastHashJoin là lựa chọn tối ưu.

### 5.5.3 Đếm Exchange Nodes (Shuffle)

Dựa trên physical plan thực tế, các shuffle Exchange chính gồm:

| # | Exchange                                 | Mục đích                                                    |
|---|------------------------------------------|-------------------------------------------------------------|
| 1 | hashpartitioning(promo_id)               | Gom ngày xuất hiện của từng promotion để tính min/max date. |
| 2 | hashpartitioning(order_row_id, promo_id) | Khử trùng lặp cặp order-promotion trước khi countDistinct.  |
| 3 | hashpartitioning(order_row_id)           | Gom các promotion hợp lệ theo từng order.                   |
| 4 | hashpartitioning(ship_state_norm)        | Tính trung bình Amount cho Merchant+Shipped theo state.     |
| 5 | hashpartitioning(ship_city_norm)         | Gom các đơn qualified theo city để tính percentage.         |
| 6 | rangepartitioning(ship-city)             | Sort toàn cục output cuối.                                  |

**Lưu ý:** Plan còn có các BroadcastExchange cho promoValidity, validPromotionCounts và stateAvgDf; đây là trao đổi broadcast, không phải shuffle hash/range của bảng lớn.

#### 5.5.4 Dếm Stages

Mỗi shuffle Exchange tạo một ranh giới stage mới. Với 6 shuffle Exchange chính và 3 BroadcastExchange, pipeline kết quả có khoảng **7–9 stages** cho action ghi output; số stage thực tế có thể thay đổi vì trước đó chương trình còn các action count() và show() phục vụ logging/report.

1. **Stage 1:** Đọc CSV, parse date/amount, scan toàn bộ input.
2. **Stage 2:** Explode promotion-ids, shuffle theo promo\_id, tính min/max date.
3. **Stage 3:** Filter Merchant+Shipped, shuffle theo ship\_state\_norm, tính avg Amount.
4. **Stage 4:** Explode order promotions, join broadcast với promoValidity, shuffle theo order\_row\_id.
5. **Stage 5:** Filter Standard, BroadcastHashJoin với validPromotionCounts.
6. **Stage 6:** BroadcastHashJoin với stateAvgDf, filter Amount.
7. **Stage 7:** Shuffle theo ship\_city\_norm, tính count/cancelled.
8. **Stage 8:** Sort toàn cục, coalesce và ghi Parquet.

### 5.5.5 Thảo luận Adaptive Query Execution (AQE)

Trong source Task 2.1, nhóm không cấu hình `spark.sql.adaptive.enabled` một cách tường minh; chương trình dùng giá trị mặc định của Spark runtime. Với môi trường kiểm thử Spark 3.4.0, physical plan in ra `AdaptiveSparkPlan`, nghĩa là AQE đang hoạt động. AQE có thể:

- **Coalesce shuffle partitions:** gộp các partition nhỏ sau shuffle, giảm số task thừa.
- **Convert SortMergeJoin sang BroadcastHashJoin:** nếu sau filter, một bên join nhỏ hơn threshold, AQE tự chuyển strategy.
- **Optimize skewed join:** chia partition lớn thành nhiều sub-partition. Trong pipeline này, join trên `ship-state` ít bị skew vì mỗi state chỉ 1 dòng ở bằng average.

Trong plan minh họa ở trên, dòng `AdaptiveSparkPlan isFinalPlan=false` xuất hiện vì `resultDf.explain(t)` được gọi trước action ghi output. Ở thời điểm đó, Spark mới hiển thị adaptive plan ban đầu; AQE vẫn cần feedback runtime từ các shuffle stages để chốt final plan sau khi action như `write` hoặc `collect` được thực thi. Vì vậy trạng thái `isFinalPlan=false` không mâu thuẫn với việc AQE đang hoạt động, mà chỉ cho biết plan chưa được materialize hoàn toàn tại thời điểm gọi `explain`.

## 5.6 Kết quả thực thi & Xuất dữ liệu

- **Đường dẫn output:** `data/output/Task_2-1.parquet`.
- **Định dạng:** Single Parquet file, readable bằng Pandas (`pd.read_parquet`) hoặc Spark local mode.
- **Số dòng kết quả:** 2,452 city groups.
- **Kích thước file:** phụ thuộc vào metadata Parquet của lần ghi.

**Bảng tổng hợp từ file Parquet đã xuất:**

| Chỉ số                                    | Giá trị |
|-------------------------------------------|---------|
| Số dòng output                            | 2,452   |
| Số city distinct                          | 2,452   |
| Giá trị nhỏ nhất của cancelled_percentage | 0.0     |
| Giá trị lớn nhất của cancelled_percentage | 0.0     |
| Số city có cancelled_percentage > 0       | 0       |

**Sample output:**

```

1 ship-city                                cancelled_percentage
2 (CHIKMAGALUR DISTRICT).      (N.R PUR THALUKU) 0.0
3 7BARASAT                      0.0
4 ABHAYAPURI                   0.0
5 ABOHAR                      0.0
6 ABU ROAD                    0.0
7 ACHALPUR                    0.0
8 ACHANTA                     0.0
9 ADAJAN GAM, SURAT           0.0
10 ADALAJ                     0.0
11 ADBHAR                     0.0

```

**Kiểm chứng thủ công.** Chọn ship-city = BENGALURU. Sau khi áp dụng đủ ba điều kiện, truy vấn kiểm chứng thu được 1,318 order qualified, trong đó cancelled\_orders = 0. Các dòng mẫu đều có ship-service-level = Standard, valid\_promo\_count >= 3, và Amount nhỏ hơn trung bình Merchant+Shipped của state tương ứng. Do đó:

$$\text{cancelled\_percentage} = \frac{0}{1318} \times 100 = 0.0000$$

Giá trị này khớp với output BENGALURU, 0.0. Toàn bộ output Task 2.1 trong lần chạy hiện tại đều có tỷ lệ 0 vì không có order qualified nào mang trạng thái Cancelled.

## 5.7 Minh chứng bằng hình ảnh

```
=====
TASK 2.1 RUN SUMMARY
=====
[Task 2-1] Loaded 128975 rows from data/input/Amazon Sale Report.csv
[Task 2-1] Found 185 temporally-valid promotions
[Task 2-1] After Cond 1+2: 32303 orders
[Task 2-1] After all 3 conditions: 17047 qualifying orders
[Task 2-1] Result preview:
+-----+-----+
|ship-city|cancelled_percentage|
+-----+-----+
|(CHIKMAGALUR DISTRICT). (N.R PUR THALUKU)|0.0|
|7BARASAT|0.0|
|ABHAYAPURI|0.0|
|ABOHAR|0.0|
|ABU ROAD|0.0|
+-----+-----+
[Task 2-1] Total cities in result: 2452
[Task 2-1] Output written to data/output/Task_2-1.parquet
[Task 2-1] Completed successfully.
```

Hình 7: Kết quả chạy chương trình Task 2.1

## BÁO CÁO LAB 3

```
May 19 6:56 PM en 100%
pearspringmind@master:~/Studying/Big Data/Lab03/Lab03_BigData

[Task 2.1] Total cities in result: 2947
=====
EXECUTION PLAN - explain(true)
=====
== Parsed Logical Plan ==
'sort ['ship-state ASC NULLS FIRST, 'ship-city ASC NULLS FIRST], true
+- Project [ship-city#33, ship-state#3090, total_orders#4555L, cancelled_orders#4557L, round(((cast(cancelled_orders#4557L as double) / cast(total_orders#4555L as double)) * 100.0), 4) AS cancelled_percentage#4562]
  +- Aggregate [ship-city#33, ship-state#3090], [ship-city#33, ship-state#3090, count(1) AS total_orders#4555L, sum(CASE WHEN (lower(trim(Status#20, None)) = cancelled) THEN 1 ELSE 0 END) AS cancelled_orders#4557L]
    +- Filter (Amount#92 < merchant_shipped_avg#2403)
      +- Filter isnotnull(Amount#92)
        +- Project [index#17, Order ID#18, Date#19, Status#20, Fulfillment#21, Sales Channel #22, ship-service-level#23, Style#24, SKU#25, Category#26, Size#27, ASIN#28, Courier Status#29, Qty#30, currency#31, Amount#92, ship-city#33, ship-postal-code#35, ship-country#36, promotion-ids#37, B2B#38, fulfilled-by#39, Unnamed: 22#40, parsed_date#65, ... 4 more fields]
          +- Join Inner, (ship-state#34 = ship-state#3090)
            +- Filter (valid_promo_count#1603 >= 3)
              +- Project [index#17, Order ID#18, Date#19, Status#20, Fulfillment#21, Sales Channel #22, ship-service-level#23, Style#24, SKU#25, Category#26, Size#27, ASIN#28, Courier Status#29, Qty#30, currency#31, Amount#92, ship-city#33, ship-postal-code#35, ship-country#36, promotion-ids#37, B2B#38, fulfilled-by#39, Unnamed: 22#40, ... 3 more fields]
                +- Project [index#17, Order ID#18, Date#19, Status#20, Fulfillment#21, Sales Channel #22, ship-service-level#23, Style#24, SKU#25, Category#26, Size#27, ASIN#28, Courier Status#29, Qty#30, currency#31, cast(Amount#32 as double) AS Amount#92, ship-city#33, ship-state#34, ship-postal-code#35, ship-country#36, promotion-ids#37, B2B#38, fulfilled-by#39, Unnamed: 22#40, parsed_date#65]
                  +- Project [index#17, Order ID#18, Date#19, Status#20, Fulfillment#21, Sales Channel #22, ship-service-level#23, Style#24, SKU#25, Category#26, Size#27, ASIN#28, Courier Status#29, Qty#30, currency#31, Amount#32, ship-city#33, ship-state#34, ship-postal-code#35, ship-country#36, promotion-ids#37, B2B#38, fulfilled-by#39, Unnamed: 22#40, to_date(Date#19, Some('MM-dd-yy'), Some(Asia/Ho_Chi_Minh)) AS parsed_date#65]
                    +- Relation [index#17,Order ID#18,Date#19,Status#20,Fulfillment#21,Sales Channel #22,ship-service-level#23,Style#24,SKU#25,Category#26,Size#27,ASIN#28,Courier Status#29,Qty#30,currency#31,Amount#32,ship-city#33,ship-postal-code#35,ship-country#36,promotion-ids#37,B2B#38,fulfilled-by#39,Unnamed: 22#40] csv
                      +- Aggregate [ship-state#3090], [ship-state#3090, avg(Amount#92) AS merchant_shipped_avg#2403]
                        +- Filter isnotnull(Amount#92)
                          +- Filter ((lower(trim(Fulfillment#3077, None)) = merchant) AND (lower(trim(Courier Status#3085, None)) = shipped))
                            +- Project [index#3073, Order ID#3074, Date#3075, Status#3076, Fulfillment#3077, Sales Channel #3078, ship-service-level#3079, Style#3080, SKU#3081, Category#3082, Size#3083, ASIN#3084, Courier Status#3085, Qty#3086, currency#3087, Amount#92, ship-city#3089, ship-state#3090, ship-postal-code#3091, ship-country#3092, promotion-ids#3093, B2B#3094, fulfilled-by#3095, Unnamed: 22#3096, ... 2 more fields]
                              +- Project [index#3073, Order ID#3074, Date#3075, Status#3076, Fulfillment#3077, Sales Channel #3078, ship-service-level#3079, Style#3080, SKU#3081, Category#3082, Size#3083, ASIN#3084, Courier Status#3085, Qty#3086, currency#3087, cast(Amount#3088 as double) AS Amount#92, ship-city#3089, ship-state#3090, ship-postal-code#3091, ship-country#3092, promotion-ids#3093, B2B#3094, fulfilled-by#3095, Unnamed: 22#3096, to_date(Date#3075, Some('MM-dd-yy'), Some(Asia/Ho_Chi_Minh)) AS parsed_date#65]
                                +- Relation [index#3073,Order ID#3074,Date#3075,Status#3076,Fulfillment#3077,Sales Channel #3078,ship-service-level#3079,Style#3080,SKU#3081,Category#3082,Size#3083,ASIN#3084,Courier Status#3085,Qty#3086,currency#3087,Amount#3088,ship-city#3089,ship-state#3090,ship-postal-code#3091,ship-country#3092,promotion-ids#3093,B2B#3094,fulfilled-by#3095,Unnamed: 22#3096] csv
                                  +- Analyzed Logical Plan ==
                                    ship-city: string, ship-state: string, cancelled_percentage: double
                                    Sort [ship-state#3090 ASC NULLS FIRST, ship-city#33 ASC NULLS FIRST], true
                                  +- Project [ship-city#33, ship-state#3090, cancelled_percentage#4562]
                                    +- Aggregate [ship-city#33, ship-state#3090], [ship-city#33, ship-state#3090, count(1) AS total_orders#4555L, sum(CASE WHEN (lower(trim(Status#20, None)) = cancelled) THEN 1 ELSE 0 END) AS cancelled_orders#4557L]
                                      +- Filter (Amount#92 < merchant_shipped_avg#2403)
                                        +- Filter isnotnull(Amount#92)
                                          +- Project [index#17, Order ID#18, Date#19, Status#20, Fulfillment#21, Sales Channel #22, ship-service-level#23, Style#24, SKU#25, Category#26, Size#27, ASIN#28, Courier Status#29, Qty#30, currency#31, Amount#92, ship-city#33, ship-postal-code#35, ship-country#36, promotion-ids#37, B2B#38, fulfilled-by#39, Unnamed: 22#40, parsed_date#65, ... 4 more fields]
                                            +- Join Inner, (ship-state#34 = ship-state#3090)
                                              +- Filter (valid_promo_count#1603 >= 3)
                                                +- Project [index#17, Order ID#18, Date#19, Status#20, Fulfillment#21, Sales Channel #22, ship-service-level#23, Style#24, SKU#25, Category#26, Size#27, ASIN#28, Courier Status#29, Qty#30, currency#31, Amount#92, ship-city#33, ship-
```

Hình 8: Execution plan của Task 2.1

```
=====
TASK 2.1 PARQUET OUTPUT PREVIEW (Task_2-1.parquet)
=====
+-----+-----+
|ship-city|cancelled_percentage|
+-----+-----+
|(CHIKMAGALUR DISTERICT). (N.R PUR THALUKU)|0.0|
|7BARASAT|0.0|
|ABHAYAPURI|0.0|
|ABOHAR|0.0|
|ABU ROAD|0.0|
|ACHALPUR|0.0|
|ACHANTA|0.0|
|ADAJAN GAM, SURAT|0.0|
|ADALAJ|0.0|
|ADBHAR|0.0|
|ADDANKI|0.0|
|ADDANKI MANDAL BAPATL DISTRICT|0.0|
|ADILABAD|0.0|
|ADIMALY|0.0|
+-----+-----+
only showing top rows

Total records in Task 2.1 output: 2452
Distinct cities: 2452
Min/Max cancelled_percentage: 0.0 / 0.0
Rows with cancelled_percentage > 0: 0
```

Hình 9: Một phần nội dung file Task\_2-1.parquet

## 6 Task 2.2: StdDev of Amount by SKU-Month with Percentile Threshold

### 6.1 Phân tích truy vấn (Query Analysis)

Task 2.2 được phụ trách bởi Nguyễn Hồ Anh Tuấn (23120185). Mục tiêu của truy vấn là xét từng nhóm (SKU, month), tìm các order có số lượng promotion đủ cao theo ngưỡng percentile động của chính nhóm đó, sau đó tính độ lệch chuẩn tổng thể của Amount trên tập order được giữ lại. Hai mức percentile bắt buộc là P90 và P80.

Đơn vị lọc không phải là **Amount**, mà là số promotion của từng order. Cột **promotion-ids** được tách theo dấu phẩy, trim từng token, bỏ token rỗng và đếm tất cả promotion id còn lại. Các promotion do Amazon phát hành vẫn được tính như promotion thông thường theo yêu cầu đề bài. Nếu **promotion-ids** null hoặc rỗng, **promo\_count** của order được đặt bằng 0.

Với mỗi nhóm **SKU-month**, chương trình tính hai threshold: **p90\_thresh** và **p80\_thresh**. Một order được xem là qualify cho mức P90 nếu **promo\_count**  $\geq$  **p90\_thresh**; tương tự, qualify cho P80 nếu **promo\_count**  $\geq$  **p80\_thresh**. Threshold được tính riêng cho từng nhóm, không dùng một ngưỡng toàn dataset.

Độ lệch chuẩn được tính bằng **stddev\_pop**, tức population standard deviation với degree of freedom bằng 0:

$$\sigma = \sqrt{\frac{\sum_{i=1}^N (x_i - \mu)^2}{N}}$$

Nếu sau khi lọc theo percentile có ít hơn 2 order, kết quả stddev của nhóm đó được đặt bằng 0.0.

Quy tắc này tránh diễn giải sai các nhóm chỉ còn 0 hoặc 1 order sau lọc.

Output được ghi thành một file Parquet duy nhất **data/output/Task\_2-2.parquet** với schema:

| Cột               | Ý nghĩa                                          |
|-------------------|--------------------------------------------------|
| SKU               | Mã SKU của sản phẩm                              |
| month             | Tháng theo định dạng yyyy-MM                     |
| p90_stddev_approx | Stddev Amount khi dùng P90 approximate threshold |
| p90_stddev_exact  | Stddev Amount khi dùng P90 exact threshold       |
| p80_stddev_approx | Stddev Amount khi dùng P80 approximate threshold |
| p80_stddev_exact  | Stddev Amount khi dùng P80 exact threshold       |

## 6.2 Phân rã thành các bước cơ bản (Decomposition)

File nguồn của phần này là **src/Task\_2-2/source/Task2\_2.scala**. Toàn bộ lời giải dùng Spark Structured APIs bằng Scala, không dùng Spark SQL dạng chuỗi truy vấn.

1. **Đọc CSV và chọn cột cần thiết.** Chương trình đọc **data/input/Amazon Sale Report.csv** với header, giữ các cột **SKU**, **Date**, **promotion-ids**, **Amount**. Cột **Amount** được cast sang **DoubleType**; các dòng không có SKU, không có ngày, hoặc không cast được **Amount** bị loại để đảm bảo phép tính stddev hợp lệ.

2. **Chuẩn hóa tháng.** Cột Date được parse bằng nhiều định dạng phổ biến như M/d/yyyy, MM-dd-yy, M/d/yy. Sau khi parse thành ngày hợp lệ, chương trình tạo `month = date_format(parsed_date, "yyyy-MM")`.
3. **Tính promo\_count.** Cột `promotion-ids` được split theo dấu phẩy, trim từng phần tử và filter token rỗng. Số token còn lại là `promo_count`; null hoặc chuỗi rỗng được đặt bằng 0.
4. **Tạo base DataFrame.** `baseDf` chỉ giữ `SKU`, `month`, `promo_count`, `amount_val`. DataFrame này được cache vì cả hai approach approximate và exact đều dùng lại cùng một tập dữ liệu đầu vào.
5. **Tính approximate thresholds.** `approxThresholds` group theo `SKU`, `month` và tính P90/P80 bằng `percentile_approx(promo_count, p, 10000)`. Threshold được join ngược lại với `baseDf` để lọc order qualify.
6. **Tính exact thresholds.** Chương trình dùng `window partitionBy("SKU", "month").orderBy(promo_count)` để gán `row_number`. Với mỗi nhóm có  $N$  order, vị trí percentile được xác định theo nearest-rank:  $\lceil p \times N \rceil$ . Giá trị `promo_count` tại vị trí đó là exact threshold.
7. **Tính stddev cho từng mức percentile.** Các order có `promo_count >= threshold` được group theo `SKU`, `month`. Hàm helper `computeStddev` đếm số order qualify và tính `stddev_pop(amount_val)` nếu count nhỏ hơn 2 thì trả 0.0.
8. **Ghép kết quả cuối.** Các kết quả P90/P80 của approximate và exact được left join vào danh sách tất cả `SKU-month` trong `baseDf`. Các giá trị null được thay bằng 0.0 và output giữ full precision của `stddev_pop`.
9. **Ghi single Parquet.** Hàm `writeParquetOutput` dùng `coalesce(1)` để tạo một part file tạm, sau đó rename part file thành đúng tên `Task_2-2.parquet`. File đầu ra đọc được bằng Pandas hoặc Spark local mode trên filesystem thông thường.

## 6.3 Lý do phân rã

Bước tính `promo_count` phải đứng trước percentile vì percentile của bài toán không được tính trên `Amount`. Percentile được tính trên số promotion của từng order, còn `Amount` chỉ được dùng ở bước cuối để tính độ lệch chuẩn của các order đã qua điều kiện lọc.

Threshold là thông tin ở cấp nhóm **SKU-month**, trong khi quyết định một order có qualify hay không là thông tin ở cấp dòng. Vì vậy sau khi aggregate để lấy threshold, chương trình phải join threshold ngược lại với **baseDf**. Cách phân rã này giữ rõ ranh giới giữa phép tính cấp nhóm và phép lọc cấp order.

Hai approach percentile được triển khai để đáp ứng đúng yêu cầu so sánh của đề bài. Approach approximate tận dụng aggregation built-in **percentile\_approx**, thường nhanh hơn vì không cần sort đầy đủ từng nhóm. Approach exact dùng window, sort và nearest-rank nên tốn chi phí hơn, nhưng threshold là giá trị xác định chính xác theo quy ước  $[p \times N]$  và luôn là một **promo\_count** quan sát được trong nhóm.

## 6.4 Chiến lược thực thi & Triển khai mã nguồn

Chương trình được chạy trực tiếp bằng **spark-shell** theo cấu trúc runner của repo:

```
1 :load src/Task_2-2/source/Task2_2.scala
2 Task2_2.main(Array(
3   "data/input/Amazon Sale Report.csv",
4   "data/output/Task_2-2.parquet"
5 ))
```

SparkSession trong source dùng local mode, đặt **spark.sql.shuffle.partitions = 8** và bật **spark.sql.adaptive.enabled = true**. Cấu hình này phù hợp với dữ liệu lab chạy local: số partition shuffle không quá lớn để tránh overhead, đồng thời AQE có thể tự tối ưu các shuffle nhỏ.

Phần chuẩn bị dữ liệu trong code tập trung vào việc tạo **baseDf**:

```
1 val baseDf = withMonthDf
2   .withColumn("promo_count",
3     when(col("promotion-ids").isNull
4       || trim(col("promotion-ids")) === "",
5       lit(0L))
6     .otherwise(
7       size(filter(
8         split(col("promotion-ids"), ","),
9         (token: Column) => trim(token) != lit("")
10       )).cast(LongType)))
11   .select("SKU", "month", "promo_count", "amount_val")
```

Approach approximate tính cả P90 và P80 trong cùng một lần groupBy:

```
1 val approxThresholds = baseDf
2   .groupBy("SKU", "month")
3   .agg(
4     percentile_approx(col("promo_count"),
5       lit(0.9), lit(10000)).alias("p90_thresh"),
6     percentile_approx(col("promo_count"),
7       lit(0.8), lit(10000)).alias("p80_thresh"))
```

Approach exact dùng window để lấy vị trí nearest-rank:

```
1 val rankedDf = baseDf
2   .withColumn("rn",
3     row_number().over(winGroupOrd))
4   .withColumn("total_n",
5     count("*").over(winGroup))
6
7 val withPosDf = rankedDf
8   .withColumn("pos_p90",
9     ceil(lit(0.9) * col("total_n")).cast(LongType))
10  .withColumn("pos_p80",
11    ceil(lit(0.8) * col("total_n")).cast(LongType))
```

Kết quả cuối được assemble từ tất cả nhóm SKU-month để không mất nhóm hợp lệ nào:

```
1 val resultDf = allGroups
2   .join(p90ApproxDf, Seq("SKU", "month"), "left")
3   .join(p80ApproxDf, Seq("SKU", "month"), "left")
4   .join(p90ExactDf, Seq("SKU", "month"), "left")
5   .join(p80ExactDf, Seq("SKU", "month"), "left")
6   .select("SKU", "month",
7     "p90_stddev_approx", "p90_stddev_exact",
8     "p80_stddev_approx", "p80_stddev_exact")
```

## 6.5 Phân tích chuyên sâu

### 6.5.1 Thuật toán bên trong `approx_percentile`

Hàm `percentile_approx` (hay `approx_percentile`) trong Spark sử dụng thuật toán **Greenwald-Khanna** (GK). Thuật toán này duy trì một cấu trúc dữ liệu nén (summary) cho phép ước lượng quantile bất kỳ trên luồng dữ liệu với guarantee:

$$\text{rank}_{\text{exact}}(q) - \epsilon \cdot N \leq \text{rank}_{\text{approx}}(q) \leq \text{rank}_{\text{exact}}(q) + \epsilon \cdot N$$

trong đó  $\epsilon = 1/\text{accuracy}$ . Với `accuracy = 10000`, sai số tương đối tối đa là  $\frac{1}{10000} = 0.01\%$ . Thuật toán GK có các đặc điểm:

- **Single-pass:** chỉ cần duyệt dữ liệu một lần, không cần sort toàn bộ.
- **Bộ nhớ:**  $O(\frac{1}{\epsilon} \log(\epsilon N))$ , nhỏ hơn nhiều so với  $O(N)$  cho exact sort.
- **Mergeable:** summary từ nhiều partition có thể merge — phù hợp với kiến trúc shuffle của Spark.

### 6.5.2 Exact percentile: sort + row\_number

Approach exact sử dụng **nearest-rank method** qua window function:

1. Partition dữ liệu theo (SKU, month), sort theo `promo_count` tăng dần.
2. Gán `row_number` (1-indexed) cho mỗi order trong nhóm.
3. Tính `pos = ⌈p × N⌉` — vị trí percentile theo nearest-rank.
4. Lọc dòng có `rn == pos` để lấy threshold.

Độ phức tạp:  $O(N \log N)$  cho mỗi nhóm SKU-month (sort partition). Tổng cộng trên toàn dataset:  $O(\sum_g N_g \log N_g)$  với  $g$  là các nhóm. Đây là chi phí cao hơn GK ( $O(N)$  amortized), nhưng đảm bảo threshold chính xác tuyệt đối.

**Lý do chọn nearest-rank thay vì interpolation:** Nearest-rank guarantee rằng percentile value luôn là một giá trị thực tế đã quan sát trong dữ liệu (không nội suy). Điều này nhất quán với cách `approx_percentile` hoạt động, giúp so sánh hai approach công bằng hơn.

**6.5.3 Khi nào `approx_percentile` đủ tốt, khi nào `exact` bắt buộc**

| <code>approx_percentile</code> phù hợp                                          | <code>exact_percentile</code> cần thiết                                                                                 |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Nhóm lớn ( $N > 100$ ): sai số $\epsilon N$ bé so với khoảng cách giữa các rank | Nhóm nhỏ ( $N < 10$ ): sai số $\epsilon N$ có thể trùng rank liền kề, dẫn đến threshold lệch                            |
| <code>promo_count</code> phân bố đều: nhiều giá trị distinct, ít tied ranks     | <code>promo_count</code> skewed/tied: nhiều order có cùng count, threshold lệch 1 đơn vị có thể thay đổi qualifying set |
| Mục đích exploration, phân tích nhanh                                           | Audit, tái lập kết quả chính xác, so sánh với baseline                                                                  |

Ảnh hưởng đến correctness của stddev: nếu threshold approximate khác exact, tập order qualify sẽ khác, dẫn đến mean và variance khác. Trong trường hợp `promo_count` có nhiều tied values (rất phổ biến trong dataset này, vì đa số orders có 0–3 promotions), việc lệch threshold 1 đơn vị có thể thêm hoặc bớt một lượng lớn orders, làm stddev thay đổi đáng kể.

**6.6 So sánh hai phương pháp Percentile**

Benchmark được đo 5 lần trên cùng `baseDf` đã cache. Mỗi lần đo bao gồm phần tính threshold, join threshold lại dữ liệu gốc, lọc order qualify, tính stddev và kích hoạt action tổng hợp. Dữ liệu sau khi lọc `Amount` hợp lệ có 121,180 order và 16,345 nhóm `SKU-month`.

| Approach               | 5 lần chạy (s)                    | Mean (s) | Stddev (s) |
|------------------------|-----------------------------------|----------|------------|
| Approximate percentile | 2.668, 1.183, 1.057, 0.852, 0.840 | 1.3200   | 0.6862     |
| Exact percentile       | 3.781, 1.828, 1.501, 1.358, 1.391 | 1.9718   | 0.9197     |

Approximate nhanh hơn exact trong lần đo này vì `percentile_approx` là aggregation built-in, tránh việc sort đầy đủ từng `SKU-month`. Exact phải dùng window ordered theo `promo_count`, nên phát sinh chi phí sort và window nhiều hơn.

Về độ chính xác, với dataset hiện tại và `accuracy = 10000`, threshold approximate trùng exact ở cả hai mức P90 và P80:

| Metric                       | Giá trị |
|------------------------------|---------|
| Số nhóm được so sánh         | 16,345  |
| Nhóm lệch P90 threshold      | 0       |
| Nhóm lệch P80 threshold      | 0       |
| Nhóm lệch qualifying set P90 | 0       |
| Nhóm lệch qualifying set P80 | 0       |
| Dòng output lệch p90_stddev  | 0       |
| Dòng output lệch p80_stddev  | 0       |

Không có **SKU-month** nào trong dữ liệu hiện tại tạo ra tập order qualify khác nhau giữa approximate và exact. Nguyên nhân thực tế là **promo\_count** là biến rời rạc, số lượng order trong từng nhóm không quá lớn, và tham số accuracy của **percentile\_approx** được đặt cao. Trong dữ liệu lớn hơn hoặc có nhóm rất lớn, approximate vẫn có thể lệch threshold so với exact; khi đó exact đáng dùng hơn nếu yêu cầu audit hoặc tái lập tuyệt đối quan trọng hơn thời gian chạy.

## 6.7 Thảo luận về phân mảnh dữ liệu (Partitioning Strategy)

Sau khi lọc các dòng có **Amount** hợp lệ, dữ liệu có 121,180 order và 16,345 nhóm **SKU-month**. Không có nhóm nào vượt 1,000 order. Nhóm lớn nhất là **JNE3405-KR-L** trong tháng **2022-04**, gồm 381 order.

| SKU             | Month   | Số order |
|-----------------|---------|----------|
| JNE3405-KR-L    | 2022-04 | 381      |
| JNE3797-KR-L    | 2022-06 | 366      |
| SET268-KR-NP-XL | 2022-04 | 303      |
| JNE3797-KR-M    | 2022-06 | 301      |
| SET268-KR-NP-L  | 2022-04 | 300      |

Vì không có nhóm lớn hơn 1,000 order, nhóm không đủ lớn để cần một chiến lược repartition riêng theo từng key. `Manual repartition("SKU", "month")` có thể hữu ích nếu dữ liệu lớn và skew mạnh, nhưng với dữ liệu lab hiện tại, chi phí shuffle bổ sung có thể lớn hơn lợi ích.

Spark thường dùng ngưỡng partition file khoảng 128 MB cho input split. Nhóm lớn nhất chỉ có 381 order, nhỏ hơn rất xa mức dữ liệu cần tách partition riêng. Do đó cấu hình thực tế

`spark.sql.shuffle.partitions = 8` cùng AQE bật là hợp lý: đủ song song cho local mode, không tạo quá nhiều task nhỏ, và vẫn cho Spark cơ hội coalesce shuffle partition khi cần.

## 6.8 Kết quả thực thi & Xuất dữ liệu

Chương trình đã xuất thành công file `data/output/Task_2-2.parquet`. File này đọc lại được bằng Pandas với 16,345 dòng và 6 cột đúng schema. Các giá trị stddev được lưu dưới dạng Double full precision, không làm tròn trước khi ghi.

```
1 Rows: 16345
2 Columns: SKU, month,
3   p90_stddev_approx, p90_stddev_exact,
4   p80_stddev_approx, p80_stddev_exact
```

Một số dòng sample có stddev khác 0 trong output:

|   | SKU             | month   | p90_approx | p90_exact | p80_approx  | p80_exact   |
|---|-----------------|---------|------------|-----------|-------------|-------------|
| 2 | AN209-BIEGE-XL  | 2022-04 | 6.0        | 6.0       | 6.0         | 6.0         |
| 3 | BL003-50BLACK-B | 2022-04 | 0.0        | 0.0       | 3.5         | 3.5         |
| 4 | BL009-61BLACK-B | 2022-05 | 0.0        | 0.0       | 2.5         | 2.5         |
| 5 | BL013-62BLACK   | 2022-05 | 1.7496355  | 1.7496355 | 1.7496355   | 1.7496355   |
| 6 | BL021-71BLACK   | 2022-05 | 0.0        | 0.0       | 115.5290969 | 115.5290969 |

**Kiểm chứng thủ công.** Chọn nhóm AN209-BIEGE-XL, tháng 2022-04: sau khi lọc Amount hợp lệ, nhóm có 2 order. Cả hai order đều có `promo_count = 1`; danh sách Amount là  $[229.0, 241.0]$ . Với  $N = 2$ , cả P80 và P90 đều có vị trí nearest-rank là  $\lceil p \times 2 \rceil = 2$ , threshold bằng 1. Hai order đều qualify, mean Amount bằng 235.0 và:

$$\sigma = \sqrt{\frac{(229 - 235)^2 + (241 - 235)^2}{2}} = \sqrt{\frac{36 + 36}{2}} = 6.0$$

Output ghi 6.0 cho cả P90/P80 và cả approximate/exact, khớp với phép tính thủ công.

## 6.9 Minh chứng bằng hình ảnh

```
[Task2-2] Input : data/input/Amazon Sale Report.csv
[Task2-2] Output : data/output/Task_2-2.parquet
[Task2-2] Parsed rows: 121180
[Task2-2] Running Approach 1: approx_percentile ...
[Task2-2] Running Approach 2: exact percentile ...
[Task2-2] Writing output ...
[Task2-2] Done. Total (SKU, month) groups: 16345
[Task2-2] Output written to: data/output/Task_2-2.parquet
[Task2-2] Sample output (top 20 rows):
```

| SKU              | month   | p90_stddev_approx | p90_stddev_exact | p80_stddev_approx | p80_stddev_exact |
|------------------|---------|-------------------|------------------|-------------------|------------------|
| AN201-RED-M      | 2022-04 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN201-RED-M      | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN201-RED-XL     | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN201-RED-XXL    | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN202-ORANGE-M   | 2022-04 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN202-ORANGE-M   | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN202-ORANGE-S   | 2022-04 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN202-ORANGE-S   | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN202-ORANGE-XXL | 2022-04 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN203-MAROON-XXL | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN204-PURPLE-L   | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN204-PURPLE-L   | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN204-PURPLE-M   | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN204-PURPLE-S   | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN204-PURPLE-XL  | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN204-PURPLE-XXL | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN205-YELLOW-S   | 2022-04 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN205-YELLOW-XL  | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN205-YELLOW-XL  | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN205-YELLOW-XXL | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |

```
only showing top 20 rows

[pearspringmind@master Lab03_BigData]$
```

Hình 10: Kết quả chạy chương trình Task 2.2

```
=====
TASK 2.2 SAMPLE WITH NON-ZERO STANDARD DEVIATION VALUES
=====
```

| SKU             | month   | p90_stddev_approx | p90_stddev_exact | p80_stddev_approx | p80_stddev_exact |  |
|-----------------|---------|-------------------|------------------|-------------------|------------------|--|
| AN209-BIEGE-XL  | 2022-04 | 6.0               | 6.0              | 6.0               | 6.0              |  |
| BL013-62BLACK   | 2022-05 | 1.7496            | 1.7496           | 1.7496            | 1.7496           |  |
| BL021-71BLACK-B | 2022-04 | 9.4045            | 9.4045           | 9.4045            | 9.4045           |  |
| BL024-74RED-B   | 2022-05 | 2.5               | 2.5              | 2.5               | 2.5              |  |
| BL035-1616OLD   | 2022-04 | 372.0             | 372.0            | 372.0             | 372.0            |  |
| BL098-S         | 2022-06 | 35.5              | 35.5             | 35.5              | 35.5             |  |
| BL110-L         | 2022-06 | 53.19             | 53.19            | 53.19             | 53.19            |  |
| BL113-S         | 2022-04 | 213.4753          | 213.4753         | 213.4753          | 213.4753         |  |
| BTM042-PP-XL    | 2022-04 | 8.0               | 8.0              | 8.0               | 8.0              |  |
| CH213-BROWN-S   | 2022-06 | 150.5             | 150.5            | 150.5             | 150.5            |  |
| J0002-SKD-L     | 2022-05 | 424.4278          | 424.4278         | 424.4278          | 424.4278         |  |
| J0002-SKD-M     | 2022-04 | 22.1599           | 22.1599          | 22.1599           | 22.1599          |  |
| J0002-SKD-M     | 2022-05 | 5.0               | 5.0              | 29.5836           | 29.5836          |  |
| J0002-SKD-XL    | 2022-05 | 32.4875           | 32.4875          | 32.4875           | 32.4875          |  |
| J0002-SKD-XL    | 2022-06 | 14.9943           | 14.9943          | 14.9943           | 14.9943          |  |
| J0002-SKD-XS    | 2022-05 | 33.4697           | 33.4697          | 33.4697           | 33.4697          |  |
| J0002-SKD-XXL   | 2022-05 | 593.0             | 593.0            | 343.1485          | 343.1485         |  |
| J0003-SET-L     | 2022-04 | 2.2111            | 2.2111           | 1.8325            | 1.8325           |  |
| J0003-SET-L     | 2022-05 | 19.3276           | 19.3276          | 15.2798           | 15.2798          |  |
| J0003-SET-S     | 2022-04 | 10.1666           | 10.1666          | 8.3927            | 8.3927           |  |

```
only showing top 20 rows
```

Hình 11: Benchmark giữa approximate và exact percentile

```
=====
TASK 2.2 PARQUET OUTPUT PREVIEW (Task_2-2.parquet)
=====
```

| SKU              | month   | p90_stddev_approx | p90_stddev_exact | p80_stddev_approx | p80_stddev_exact |
|------------------|---------|-------------------|------------------|-------------------|------------------|
| AN201-RED-M      | 2022-04 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN201-RED-M      | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN201-RED-XL     | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN201-RED-XXL    | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN202-ORANGE-M   | 2022-04 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN202-ORANGE-M   | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN202-ORANGE-S   | 2022-04 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN202-ORANGE-S   | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN202-ORANGE-XXL | 2022-04 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN203-MAROON-XXL | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN204-PURPLE-L   | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN204-PURPLE-L   | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN204-PURPLE-M   | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN204-PURPLE-S   | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN204-PURPLE-XL  | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN204-PURPLE-XXL | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN205-YELLOW-S   | 2022-04 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN205-YELLOW-XL  | 2022-05 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN205-YELLOW-XL  | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |
| AN205-YELLOW-XXL | 2022-06 | 0.0               | 0.0              | 0.0               | 0.0              |

```
only showing top 20 rows
```

Hình 12: Một phần nội dung file Task\_2-2.parquet

## 7 Kết luận

### 7.1 Tổng kết kết quả

Nhóm đã hoàn thành cả bốn task của Lab 03, tạo ra bốn file output đúng định dạng yêu cầu:

- **Task 1-1:** Task\_1-1.csv — 3,543 dòng kết quả, xác định size được mua nhiều nhất theo sliding window 7 ngày cho mỗi state.
- **Task 1-2:** Task\_1-2.csv — 128 dòng kết quả, tính median variety cấp bang theo tháng cho các style có size  $\geq$  XXL.
- **Task 2-1:** Task\_2-1.parquet — 2,452 dòng kết quả, tỷ lệ đơn cancelled theo city trên tập đơn thỏa ba điều kiện đồng thời.

- **Task 2-2:** Task\_2-2.parquet — population stddev của Amount theo SKU-month với ngưỡng percentile P90/P80, hai approach approximate và exact.

Mỗi task đã được kiểm chứng thủ công bằng cách chọn mẫu từ output, đối chiếu ngược lại dataset gốc và tính toán lại kết quả.

## 7.2 Trade-off giữa MapReduce và Spark Structured APIs

| Tiêu chí               | MapReduce (Task 1-x)                                                               | Spark Structured API (Task 2-x)                                                            |
|------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Abstraction level      | Thấp — lập trình viên kiểm soát key-value, emit logic, reducer                     | Cao — declarative transformations, Catalyst tự tối ưu plan                                 |
| Ease of implementation | Code dài hơn (~250 dòng Task 1-1, ~125 dòng Task 1-2) nhưng luồng xử lý tường minh | Code ngắn gọn hơn cho logic phức tạp; join/filter/aggregation chỉ vài dòng                 |
| Tối ưu hóa             | Thủ công: chọn key design, quyết định dùng reduceByKey vs groupByKey               | Tự động: Catalyst chọn join strategy, AQE điều chỉnh runtime                               |
| Phù hợp với bài toán   | Sliding window, median — cần kiểm soát chi tiết emission pattern và sort logic     | Multi-condition filter, join state average, percentile — cần compose nhiều transformations |

## 7.3 Bài học rút ra

1. **Sliding window ↔ emission pattern:** Thay vì quét ngược lại quá khứ cho mỗi ngày tính toán, phương pháp “đẩy xuôi” (emit đóng góp cho các ngày tương lai) biến sliding window thành bài toán gom nhóm key-value, hoàn toàn phù hợp với MapReduce.
2. **Median đòi hỏi full sort:** Không thể dùng Combiner hay reduceByKey cho median vì

median không có tính associative. Bài học: xác định rõ tính chất toán học của aggregation trước khi chọn API.

3. **Broadcast join cho bảng nhỏ:** Trong Task 2-1, bảng state average (~36 dòng) nhỏ đủ để broadcast, tránh shuffle trên bảng orders lớn. Nguyên tắc: luôn kiểm tra kích thước bảng join để chọn strategy phù hợp.
4. **Approximate vs exact percentile:** Task 2-2 cho thấy `approx_percentile` (Greenwald-Khanna) nhanh hơn nhưng có thể cho threshold khác exact (đặc biệt trên nhóm nhỏ hoặc phân bố skewed). Exact percentile qua `row_number + ceil(p × N)` đảm bảo chính xác nhưng tốn shuffle + sort.
5. **Chuẩn hóa dữ liệu:** Dữ liệu thực tế có nhiều vấn đề: khoảng trắng thừa, chữ hoa/thường không nhất quán, null values, format ngày đa dạng. Việc `trim + lower/upper` ngay đầu pipeline là bắt buộc để tránh sai lệch kết quả.

## 7.4 Khó khăn và hướng cải thiện

### Khó khăn gặp phải:

- Parse ngày tháng: dataset chứa nhiều format date khác nhau, cần cascade qua nhiều pattern.
- Promotion-ids: chuỗi comma-separated với khoảng trắng không đều, trailing comma tạo token rỗng.
- Cấu hình Spark trên Linux native: cần cố định đúng bản Spark/Scala/Java và đặt `SPARK_LOCAL_IP`, `SPARK_LOCAL_HOSTNAME` để đảm bảo driver bind ổn định ở local mode.

### Hướng cải thiện:

- Bổ sung unit test tự động cho từng bước transformation, đối chiếu output với expected values.
- Chuẩn hóa schema input bằng một bước tiền xử lý chung cho cả bốn task.
- Tối ưu số partition dựa trên kích thước dữ liệu thực tế thay vì dùng giá trị mặc định.

## 8 Phụ lục

### 8.1 Source code và Demo

- Link Dataset/Output: [Drive](#)
- Source code được nộp trực tiếp trong thư mục `src` của gói bài làm.

## Tài liệu

- [1] Apache Parquet. Apache parquet documentation, 2024. Columnar storage format documentation.
- [2] Apache Spark. Built-in functions, 2023. Apache Spark 3.4.0 SQL function reference.
- [3] Apache Spark. Dataset and dataframe api reference, 2023. Apache Spark 3.4.0 Scala API.
- [4] Apache Spark. Generated runtime sql configuration table, 2023. Apache Spark 3.4.0 configuration reference.
- [5] Apache Spark. Performance tuning, 2023. Apache Spark 3.4.0 documentation.
- [6] Apache Spark. Rdd programming guide, 2023. Apache Spark 3.4.0 documentation.
- [7] Apache Spark. Spark sql, dataframes and datasets guide, 2023. Apache Spark 3.4.0 documentation.
- [8] Bill Chambers and Matei Zaharia. *Spark: The Definitive Guide*. O'Reilly Media, Sebastopol, CA, USA, 2018.